

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐÔNG Á
KHOA CÔNG NGHỆ THÔNG TIN**



BÀI TẬP LỚN

HỌC PHẦN: CHUYÊN ĐỀ TỐT NGHIỆP 2

MÃ ĐỀ: 48

ĐỀ TÀI: XÂY DỰNG ỨNG DỤNG DỰ BÁO THỜI TIẾT

LỚP TÍN CHỈ: CDTN2.03.K13.07.LH.C04.1_LT

Giảng viên hướng dẫn: ThS. Trần Xuân Thanh

Danh sách sinh viên thực hiện:

STT	Mã sinh viên	Sinh viên thực hiện	Lớp hành chính
1	20223155	Nguyễn Trí Dũng	DCCNTT 13.10.16
2	20222999	Nguyễn Trung Chính	DCCNTT 13.10.16
3	20222998	Vũ Văn Phong	DCCNTT 13.10.16
4	20223053	Đào Xuân Phú	DCCNTT 13.10.16

Bắc Ninh, năm 2026

MỤC LỤC

DANH MỤC VIẾT TẮT	iv
DANH MỤC HÌNH ẢNH	v
LỜI MỞ ĐẦU	1
CHƯƠNG 1: TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT	3
1.1. Lý do chọn đề tài	3
1.2. Mục tiêu nghiên cứu	4
1.3. Phạm vi nghiên cứu	4
1.4. Phương pháp thực hiện	5
1.5. Tổng quan về dự báo thời tiết	5
1.6. Giới thiệu về Flutter	6
1.6.1. Kiến trúc của Flutter	6
1.6.2. Nguyên lý hoạt động	7
1.6.3. Đặc điểm nổi bật của Flutter	7
1.6.4. Các thành phần chính trong Flutter	8
1.6.5. So sánh Flutter với các công nghệ khác	9
1.6.6. Ứng dụng của Flutter trong đề tài	9
1.6.7. Kết luận	10
1.7. Ngôn ngữ lập trình Dart	10
1.7.1. Đặc điểm của ngôn ngữ Dart	11
1.7.2. Mô hình lập trình trong Dart	11
1.7.3. Cơ chế biên dịch của Dart	12
1.7.4. Các kiểu dữ liệu trong Dart	12
1.7.5. Lập trình bất đồng bộ trong Dart	12
1.7.6. Quản lý bộ nhớ và hiệu năng	13

1.7.7. Thư viện và hệ sinh thái Dart	13
1.7.8. Ứng dụng Dart trong đề tài	13
1.7.9. Kết luận	14
1.8. Cơ sở dữ liệu SQLite	14
1.8.1. Tổng quan về SQLite	14
1.8.2. Kiến trúc hoạt động của SQLite	15
1.8.3. Các thành phần chính trong SQLite	16
1.8.4. Các thao tác cơ bản với SQLite	16
1.8.5. Ưu điểm và nhược điểm của SQLite	16
1.8.6. Tích hợp SQLite trong Flutter	17
1.8.7. Ứng dụng SQLite trong đề tài	17
1.8.8. Kết luận	17
CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	18
2.1. Bài toán	18
2.1.1. Yêu cầu chức năng	18
2.1.2. Yêu cầu phi chức năng	19
2.2. Phân tích thiết kế hệ thống	21
2.2.1. Sơ đồ phân cấp chức năng	21
2.2.2. Sơ đồ Use Case	21
2.2.3. Sơ đồ Erd	22
2.2.4. Sơ đồ lớp	22
2.2.5. Sơ đồ tuần tự	23
2.2.6. Biểu đồ hoạt động	25
2.3. Cài đặt kiểm thử	29
2.4. Publication	31

CHƯƠNG 3: XÂY DỰNG ỨNG DỤNG	33
3.1. Môi trường phát triển	33
3.2. Cài đặt và cấu hình hệ thống	33
3.3. Xây dựng các chức năng chính	33
3.3.1. <i>Hiển thị thời tiết hiện tại</i>	33
3.3.2. <i>Dự báo thời tiết</i>	34
3.3.3. <i>Cảnh báo thời tiết</i>	34
3.3.4. <i>Lưu trữ dữ liệu bằng SQLite</i>	34
3.3.5. <i>Tích hợp API thời tiết</i>	35
3.4. Xử lý dữ liệu và logic chương trình	35
3.5. Giao diện ứng dụng	36
KẾT LUẬN	43
TÀI LIỆU THAM KHẢO	45
PHỤ LỤC	47

DANH MỤC VIẾT TẮT

STT	Chữ viết tắt	Giải thích
1	API	Application Programming Interface (Giao diện lập trình ứng dụng)
2	UI	User Interface (Giao diện người dùng)
3	UX	User Experience (Trải nghiệm người dùng)
4	DB	Database (Cơ sở dữ liệu)
5	SQL	Structured Query Language (Ngôn ngữ truy vấn có cấu trúc)
6	SQLite	Cơ sở dữ liệu nhẹ dùng cho ứng dụng di động
7	HTTP	HyperText Transfer Protocol (Giao thức truyền tải siêu văn bản)
8	JSON	JavaScript Object Notation (Định dạng dữ liệu trao đổi)
9	GPS	Global Positioning System (Hệ thống định vị toàn cầu)
10	IDE	Integrated Development Environment (Môi trường phát triển tích hợp)
11	MVC	Model - View - Controller (Mô hình kiến trúc phần mềm)
12	OOP	Object-Oriented Programming (Lập trình hướng đối tượng)
13	REST	Representational State Transfer (Kiến trúc API web)

DANH MỤC HÌNH ẢNH

Hình 1. 1: Ứng dụng flutter	6
Hình 1. 2: Kiến trúc của Flutter	7
Hình 1. 3: Đặc điểm nổi bật của Flutter	8
Hình 1. 4: Các thành phần chính trong Flutter	9
Hình 1. 5: Ngôn ngữ lập trình Dart	10
Hình 1. 6: Mô hình lập trình trong Dart	11
Hình 1. 7: Cơ sở dữ liệu SQLite	14
Hình 1. 8: Kiến trúc hoạt động của SQLite	15
Hình 2. 1: Sơ đồ phân cấp chức năng	21
Hình 2. 2: Sơ đồ Use case	21
Hình 2. 3: Sơ đồ ERD	22
Hình 2. 4: Sơ đồ lớp	22
Hình 2. 5: Biểu đồ tuần tự chọn thành phố	23
Hình 2. 6: Biểu đồ tuần tự load vị trí hiện tại	23
Hình 2. 7: Biểu đồ hoạt động thông báo thời tiết	24
Hình 2. 8: Biểu đồ tuần tự đặt lịch nhắc nhở	24
Hình 2. 9: Biểu đồ hoạt động chọn thành phố	25
Hình 2. 10: Biểu đồ hoạt động xem vị trí hiện tại	26
Hình 2. 11: Biểu đồ hoạt động gửi thông báo thời tiết	27
Hình 2. 12: Biểu đồ hoạt động đặt lịch nhắc nhở	28
Hình 2. 13: Biểu đồ hoạt động tổng quát	29
Hình 3. 1: Màn hình chính của ứng dụng part 1	36
Hình 3. 2: Màn hình chính của ứng dụng part 2	37
Hình 3. 3: Màn hình tìm kiếm thành phố	38

Hình 3. 4: Màn hình danh sách thành phố đã lưu	39
Hình 3. 5: Màn hình thiết lập nhắc nhở thời tiết	40
Hình 3. 6: Màn hình quản lý lịch nhắc cá nhân	41
Hình 3. 7: Màn hình giao diện chat AI	42

LỜI MỞ ĐẦU

Trong bối cảnh công nghệ thông tin phát triển vượt bậc, các ứng dụng di động ngày càng trở thành một phần thiết yếu trong đời sống hằng ngày. Đặc biệt, trong lĩnh vực khí tượng thủy văn, việc tiếp cận thông tin thời tiết một cách nhanh chóng, chính xác và kịp thời đóng vai trò quan trọng trong việc hỗ trợ các hoạt động sinh hoạt, sản xuất và phòng tránh các rủi ro do thiên tai gây ra (Trần & Nguyễn, 2024) [9].

Những năm gần đây, các hiện tượng thời tiết cực đoan như mưa lớn, bão, lũ lụt, nắng nóng kéo dài... diễn ra với tần suất và mức độ ngày càng gia tăng, đặt ra yêu cầu cấp thiết về các hệ thống dự báo và cảnh báo sớm. Tuy nhiên, hiện nay không phải tất cả các ứng dụng thời tiết đều đáp ứng được nhu cầu cá nhân hóa và khả năng cảnh báo kịp thời cho người dùng.

Xuất phát từ thực tiễn này, nhóm nghiên cứu đã lựa chọn đề tài: “Xây dựng ứng dụng dự báo thời tiết”. Mục tiêu của đề tài là phát triển một ứng dụng di động sử dụng nền tảng Flutter, ngôn ngữ Dart cùng cơ sở dữ liệu SQLite, nhằm cung cấp thông tin thời tiết theo thời gian thực, dự báo trong tương lai, đồng thời hiển thị cảnh báo thiên tai một cách trực quan và kịp thời.

Đề tài hướng tới việc: Cung cấp thông tin thời tiết chính xác theo thời gian thực. Dự báo các hiện tượng thiên tai, giúp người dùng chủ động phòng tránh. Xây dựng giao diện trực quan, dễ sử dụng cho người dùng cá nhân. Áp dụng kiến thức lập trình, quản lý dữ liệu và thiết kế hệ thống vào thực tiễn.

Nhóm nghiên cứu triển khai ứng dụng bằng cách: Sử dụng Flutter để xây dựng giao diện di động đa nền tảng. Lập trình xử lý dữ liệu bằng Dart, kết nối với SQLite để quản lý dữ liệu cục bộ. Tích hợp các API dự báo thời tiết để cập nhật thông tin theo thời gian thực. Thiết kế hệ thống cảnh báo dựa trên ngưỡng nguy cơ thiên tai và cơ chế thông báo trực tiếp đến người dùng.

Ứng dụng cung cấp dự báo thời tiết theo từng khu vực cụ thể, bao gồm thông tin nhiệt độ, độ ẩm, lượng mưa, gió và các hiện tượng thiên tai. Tích hợp cảnh báo sớm giúp người dùng chủ động trong sinh hoạt và công tác phòng tránh rủi ro. Tăng cường kỹ năng lập trình ứng dụng di động, xử lý dữ liệu và thiết kế hệ thống cho nhóm nghiên cứu. Mang lại giá trị thực tiễn cho cộng đồng, hỗ trợ việc theo dõi và phòng

ngừa thiên tai hiệu quả hơn.

Đề tài “Xây dựng ứng dụng dự báo thời tiết” góp phần kết nối tri thức lý thuyết với thực tiễn công nghệ, đồng thời nâng cao năng lực lập trình và thiết kế hệ thống của nhóm nghiên cứu. Mặc dù đã đạt được những kết quả tích cực, nhưng do giới hạn về thời gian và kinh nghiệm, ứng dụng vẫn còn một số hạn chế và tiềm năng cải tiến. Nhóm nghiên cứu mong muốn nhận được phản hồi từ giảng viên và chuyên gia để hoàn thiện sản phẩm trong các giai đoạn tiếp theo.

CHƯƠNG 1: TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT

1.1. Lý do chọn đề tài

Trong những năm gần đây, sự phát triển nhanh chóng của công nghệ thông tin và truyền thông đã tạo ra những bước tiến đáng kể, ảnh hưởng tích cực đến nhiều lĩnh vực trong đời sống xã hội. Các ứng dụng di động ngày càng trở nên phổ biến, đóng vai trò quan trọng trong việc giúp con người tiếp cận thông tin một cách nhanh chóng, chính xác và thuận tiện. Trong bối cảnh này, lĩnh vực dự báo thời tiết đã thu hút sự quan tâm đặc biệt từ cả cộng đồng nghiên cứu và người dùng (Nguyễn & Trần, 2023) [13].

Thời tiết là yếu tố tác động trực tiếp đến các hoạt động hằng ngày, từ việc đi học, đi làm, đến các hoạt động sản xuất nông nghiệp, vận tải, du lịch và sinh hoạt công cộng. Việc tiếp cận thông tin thời tiết kịp thời giúp con người chủ động trong việc lên kế hoạch và giảm thiểu những rủi ro không mong muốn.

Bên cạnh đó, biến đổi khí hậu toàn cầu đang diễn ra với mức độ ngày càng nghiêm trọng, dẫn đến sự gia tăng các hiện tượng thời tiết cực đoan như bão, lũ lụt, hạn hán, nắng nóng kéo dài và sạt lở đất. Những hiện tượng này không chỉ gây thiệt hại kinh tế lớn mà còn đe dọa trực tiếp đến tính mạng con người. Trong bối cảnh đó, việc xây dựng các hệ thống cảnh báo sớm trở nên cấp thiết, nhằm giảm thiểu rủi ro và bảo đảm an toàn cho cộng đồng (Trần, 2024) [14].

Mặc dù thị trường hiện nay đã có nhiều ứng dụng dự báo thời tiết, nhưng vẫn tồn tại một số hạn chế đáng chú ý:

Dữ liệu chưa được cá nhân hóa theo vị trí của người dùng.

Thiếu các cảnh báo thiên tai cụ thể và trực quan.

Giao diện chưa thân thiện hoặc khó sử dụng.

Phụ thuộc hoàn toàn vào kết nối Internet.

Xuất phát từ những hạn chế nêu trên, nhóm nghiên cứu đã quyết định lựa chọn đề tài “Xây dựng ứng dụng dự báo thời tiết” nhằm phát triển một hệ thống cung cấp thông tin thời tiết chính xác, kịp thời, đồng thời hỗ trợ cảnh báo thiên tai hiệu quả và trực quan cho người sử dụng.

Việc nghiên cứu và phát triển ứng dụng dự báo thời tiết không chỉ mang ý nghĩa thực tiễn trong việc bảo vệ con người và tài sản, mà còn là cơ hội để nhóm nghiên cứu áp dụng kiến thức về lập trình, xử lý dữ liệu và thiết kế hệ thống vào thực tiễn. Đề tài hướng tới việc cải thiện trải nghiệm người dùng, cung cấp dữ liệu cá nhân hóa và tăng cường khả năng cảnh báo sớm, góp phần nâng cao mức độ sẵn sàng đối phó với các hiện tượng thiên tai trong bối cảnh biến đổi khí hậu ngày càng phức tạp.

1.2. Mục tiêu nghiên cứu

Mục tiêu chính của đề tài là phát triển một ứng dụng di động nhằm hỗ trợ người dùng trong việc theo dõi tình hình thời tiết và nhận cảnh báo thiên tai một cách nhanh chóng, chính xác và thuận tiện. Trọng tâm của nghiên cứu là tối ưu trải nghiệm người dùng, cung cấp dữ liệu thời tiết trực quan và kịp thời, đồng thời đảm bảo hiệu suất và khả năng mở rộng của ứng dụng (Nguyễn & Trần, 2023).

Các mục tiêu cụ thể của đề tài bao gồm:

Phát triển ứng dụng di động trên nền tảng Flutter với giao diện thân thiện và hiện đại.

Hiển thị thông tin thời tiết hiện tại dựa trên vị trí thực của người dùng.

Cung cấp dự báo thời tiết trong ngắn hạn và trung hạn, bao gồm các thông số cơ bản như nhiệt độ, độ ẩm, áp suất và tốc độ gió.

Tích hợp hệ thống cảnh báo thiên tai dựa trên dữ liệu thời tiết nhằm nâng cao khả năng phòng ngừa rủi ro.

Lưu trữ dữ liệu cục bộ bằng SQLite để tối ưu hiệu năng và giảm phụ thuộc vào kết nối Internet.

Tích hợp API thời tiết để cập nhật dữ liệu theo thời gian thực, đảm bảo độ chính xác và kịp thời.

Đảm bảo ứng dụng hoạt động ổn định, dễ bảo trì và có khả năng mở rộng cho các tính năng trong tương lai.

1.3. Phạm vi nghiên cứu

Đề tài tập trung vào việc nghiên cứu và phát triển ứng dụng trên nền tảng di

động, cụ thể là:

Nền tảng phát triển: Flutter

Ngôn ngữ lập trình: Dart

Cơ sở dữ liệu: SQLite

Nguồn dữ liệu: API thời tiết (OpenWeatherMap, WeatherAPI,...)

❖ Phạm vi chức năng bao gồm:

Hiển thị thời tiết hiện tại

Dự báo thời tiết

Lưu trữ dữ liệu

Đề tài không đi sâu vào việc xây dựng mô hình dự báo thời tiết phức tạp mà sử dụng dữ liệu từ các API có sẵn.

1.4. Phương pháp thực hiện

Phương pháp nghiên cứu tài liệu: tìm hiểu lý thuyết về thời tiết, Flutter, Dart, SQLite

Phương pháp phân tích hệ thống: xác định yêu cầu và chức năng

Phương pháp thiết kế: xây dựng kiến trúc và giao diện

Phương pháp lập trình: triển khai hệ thống

Phương pháp kiểm thử: đánh giá chất lượng ứng dụng

1.5. Tổng quan về dự báo thời tiết

Dự báo thời tiết là quá trình sử dụng các dữ liệu khí tượng để dự đoán trạng thái khí quyển trong tương lai. Các yếu tố được xem xét bao gồm nhiệt độ, độ ẩm, áp suất khí quyển, tốc độ gió, lượng mưa,...

❖ Hiện nay, dự báo thời tiết sử dụng các mô hình toán học và dữ liệu từ:

Vệ tinh

Trạm quan trắc

Radar thời tiết

Dữ liệu sau khi xử lý sẽ được cung cấp thông qua các hệ thống API để phục vụ các ứng dụng.

1.6. Giới thiệu về Flutter

Flutter là một framework mã nguồn mở do **Google** phát triển, ra mắt lần đầu vào năm 2017. Framework này cho phép xây dựng các ứng dụng đa nền tảng (Android, iOS, Web và Desktop) từ một **codebase duy nhất**, giúp tối ưu thời gian phát triển và giảm thiểu chi phí bảo trì (Google, 2023). Flutter nhanh chóng trở thành một trong những công nghệ phổ biến nhất để phát triển ứng dụng di động nhờ **hiệu năng cao, giao diện đẹp và tốc độ triển khai nhanh**.

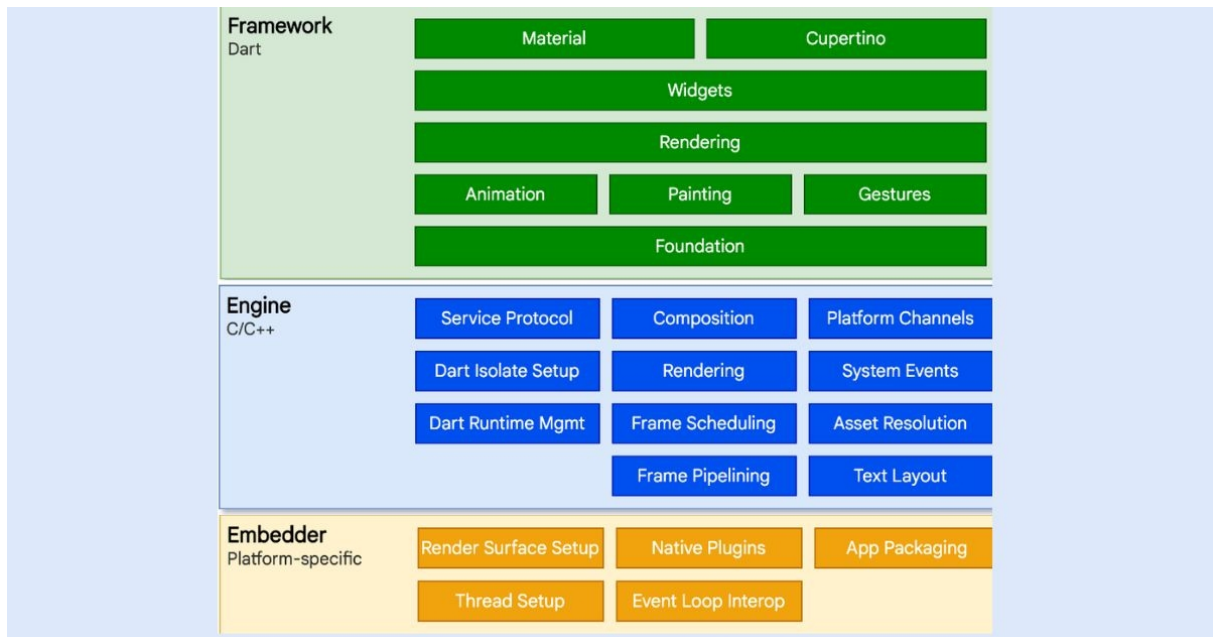


Hình 1. 1: Ứng dụng flutter

Flutter nhanh chóng trở thành một trong những công nghệ phát triển ứng dụng di động phổ biến nhờ vào hiệu năng cao, giao diện đẹp và khả năng phát triển nhanh chóng.

1.6.1. Kiến trúc của Flutter

- ❖ Flutter hoạt động dựa trên kiến trúc nhiều lớp, bao gồm:



Hình 1. 2: Kiến trúc của Flutter

Framework Layer: Cung cấp các thành phần giao diện (widget), thư viện và công cụ phát triển ứng dụng.

Engine Layer: Viết bằng ngôn ngữ C++, chịu trách nhiệm xử lý đồ họa, rendering và giao tiếp với hệ điều hành. Flutter sử dụng Skia để vẽ giao diện.

Embedder Layer: Lớp trung gian cho phép Flutter tương tác với các hệ điều hành khác nhau như Android, iOS, Windows, macOS và Linux.

Nhờ kiến trúc này, Flutter **không phụ thuộc vào các thành phần giao diện gốc của hệ điều hành**, mà tự vẽ toàn bộ UI, đảm bảo tính nhất quán trên các nền tảng.

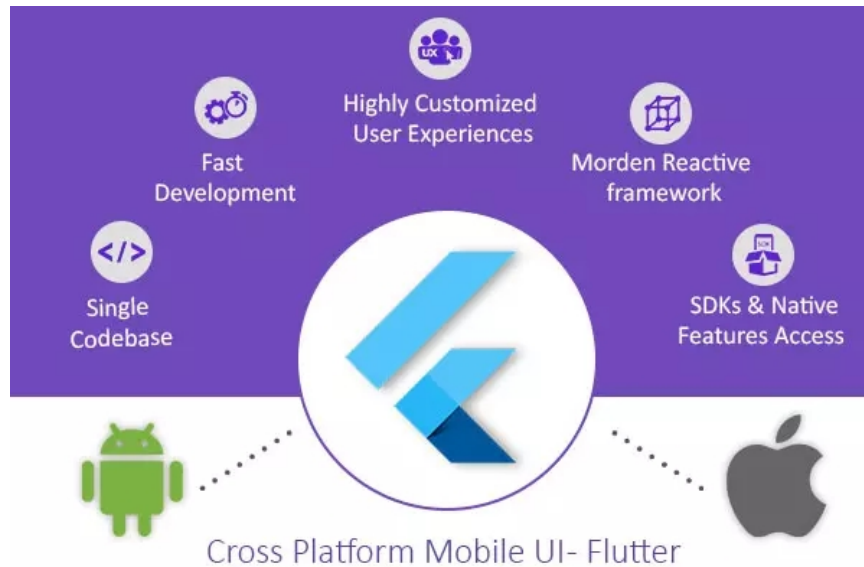
1.6.2. Nguyên lý hoạt động

Flutter sử dụng mô hình lập trình dựa trên **widget**, trong đó mọi thành phần giao diện đều là widget, từ các thành phần cơ bản như văn bản, hình ảnh, cho đến các layout phức tạp. Ứng dụng được xây dựng dưới dạng **cây widget (Widget Tree)**. Khi dữ liệu thay đổi, Flutter **cập nhật lại giao diện** thông qua cơ chế **reactive**, giúp tối ưu hiệu năng và mang lại trải nghiệm mượt mà.

Cơ chế **Hot Reload** cho phép lập trình viên cập nhật giao diện ngay lập tức mà không cần khởi động lại ứng dụng, giúp tăng tốc quá trình phát triển.

1.6.3. Đặc điểm nổi bật của Flutter

- ❖ Flutter sở hữu nhiều ưu điểm vượt trội so với các công nghệ phát triển ứng dụng truyền thống:



Hình 1. 3: Đặc điểm nổi bật của Flutter

Đa nền tảng: Viết một lần, chạy trên nhiều nền tảng.

Hiệu năng cao: Sử dụng engine riêng, không phụ thuộc bridge như React Native.

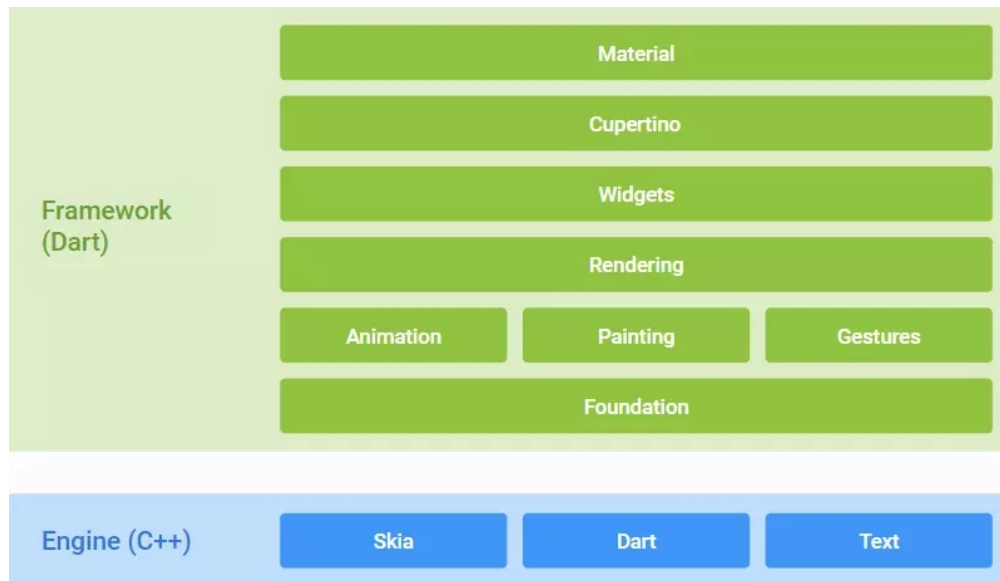
Giao diện đẹp: Hỗ trợ Material Design (Android) và Cupertino (iOS).

Hot Reload: Cập nhật giao diện nhanh trong quá trình phát triển.

Tùy biến linh hoạt: Thiết kế UI theo ý muốn mà không bị giới hạn.

1.6.4. Các thành phần chính trong Flutter

- ❖ Flutter cung cấp nhiều thành phần quan trọng hỗ trợ phát triển ứng dụng:



Hình 1. 4: Các thành phần chính trong Flutter

Widget: Thành phần cơ bản để xây dựng giao diện.

StatelessWidget: Widget không thay đổi trạng thái.

StatefulWidget: Widget có thể thay đổi trạng thái trong quá trình chạy.

BuildContext: Cung cấp thông tin về vị trí widget trong cây giao diện.

Navigator: Quản lý điều hướng giữa các màn hình.

1.6.5. So sánh Flutter với các công nghệ khác

So với các framework khác như React Native hay Native Android, Flutter có một số điểm khác biệt:

So với Native: Flutter giảm thời gian phát triển nhờ sử dụng chung codebase.

So với React Native: Flutter có hiệu năng tốt hơn do không cần bridge.

So với Web App: Flutter mang lại trải nghiệm người dùng mượt mà hơn trên thiết bị di động.

1.6.6. Ứng dụng của Flutter trong đề tài

Trong đề tài “**Xây dựng ứng dụng dự báo thời tiết**”, Flutter được lựa chọn làm nền tảng phát triển chính bởi các ưu điểm sau:

Hỗ trợ xây dựng **giao diện trực quan, dễ sử dụng**.

Dễ dàng tích hợp **API thời tiết** để lấy dữ liệu theo thời gian thực.

Tối ưu hiệu năng cho các ứng dụng di động.

Hỗ trợ phát triển nhanh và dễ bảo trì cho các phiên bản tiếp theo.

Flutter giúp nhóm triển khai ứng dụng một cách hiệu quả, đồng thời đảm bảo tính thẩm mỹ và trải nghiệm người dùng.

1.6.7. Kết luận

Flutter là một framework mạnh mẽ, hiện đại và phù hợp để phát triển ứng dụng di động trong bối cảnh công nghệ hiện nay. Với khả năng đa nền tảng, hiệu năng cao và hệ sinh thái phong phú, Flutter là lựa chọn tối ưu để xây dựng ứng dụng dự báo thời tiết, đáp ứng các yêu cầu về tốc độ phát triển, tính ổn định và trải nghiệm người dùng.

1.7. Ngôn ngữ lập trình Dart

Dart là một ngôn ngữ lập trình mã nguồn mở, được phát triển bởi Google và ra mắt lần đầu vào năm 2011. Dart được thiết kế nhằm xây dựng các ứng dụng hiện đại với hiệu năng cao, đặc biệt trong các ứng dụng giao diện người dùng trên nền tảng di động, web và desktop. Ngôn ngữ này hiện là ngôn ngữ chính trong framework Flutter, đóng vai trò trung tâm trong việc phát triển các ứng dụng đa nền tảng (Google, 2023).



Hình 1. 5: Ngôn ngữ lập trình Dart

Hiện nay, Dart là ngôn ngữ chính được sử dụng trong framework Flutter, đóng vai trò quan trọng trong việc phát triển các ứng dụng đa nền tảng.

1.7.1. Đặc điểm của ngôn ngữ Dart

Dart là một ngôn ngữ lập trình hướng đối tượng (Object-Oriented Programming – OOP), trong đó mọi thứ đều được coi là đối tượng. Dart hỗ trợ các đặc tính cơ bản của OOP như: kế thừa, đóng gói, đa hình và trừu tượng hóa.

❖ Các đặc điểm nổi bật của Dart:

Cú pháp đơn giản, dễ học, tương tự Java và C#.

Hỗ trợ cả Just-In-Time (JIT) và Ahead-Of-Time (AOT) compilation.

Tích hợp chặt chẽ với Flutter để xây dựng giao diện người dùng.

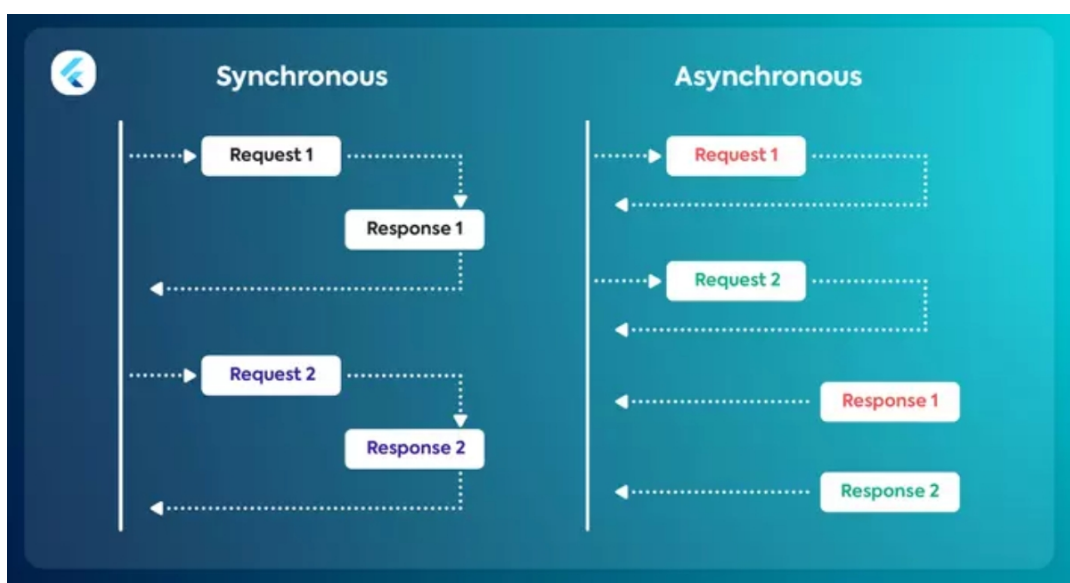
Hỗ trợ lập trình bất đồng bộ (asynchronous programming).

Quản lý bộ nhớ tự động thông qua Garbage Collection.

Nhờ những đặc điểm này, Dart giúp lập trình viên phát triển ứng dụng nhanh chóng, hiệu quả và dễ bảo trì.

1.7.2. Mô hình lập trình trong Dart

❖ Dart hỗ trợ nhiều mô hình lập trình khác nhau, trong đó phổ biến nhất là:



Hình 1. 6: Mô hình lập trình trong Dart

Hướng đối tượng (OOP): Cho phép tổ chức code thành các lớp và đối tượng, hỗ

trợ tái sử dụng và mở rộng.

Hàm (Functional Programming): Hỗ trợ các hàm bậc cao, lambda, giúp code linh hoạt hơn.

Bất đồng bộ (Asynchronous Programming): Sử dụng Future, async/await để xử lý các tác vụ như gọi API hay đọc dữ liệu mà không làm gián đoạn giao diện.

Ví dụ, khi ứng dụng cần gọi API thời tiết, Dart cho phép xử lý bất đồng bộ, đảm bảo giao diện người dùng vẫn mượt mà.

1.7.3. Cơ chế biên dịch của Dart

Just-In-Time (JIT): Hỗ trợ quá trình phát triển, cho phép Hot Reload trong Flutter.

Ahead-Of-Time (AOT): Biên dịch thành mã máy trước khi chạy, tối ưu hiệu năng khi triển khai ứng dụng.

Sự kết hợp giữa JIT và AOT giúp Dart vừa tối ưu trong quá trình phát triển, vừa đảm bảo hiệu năng khi chạy thực tế.

1.7.4. Các kiểu dữ liệu trong Dart

❖ Dart hỗ trợ nhiều kiểu dữ liệu cơ bản:

int: số nguyên

double: số thực

String: chuỗi ký tự

bool: giá trị logic (true/false)

List: danh sách

Map: cấu trúc key-value

dynamic: kiểu dữ liệu linh hoạt

Hỗ trợ đa dạng kiểu dữ liệu giúp Dart dễ dàng xử lý dữ liệu từ API, đặc biệt là dữ liệu JSON.

1.7.5. Lập trình bất đồng bộ trong Dart

Trong các ứng dụng hiện đại, đặc biệt là ứng dụng di động, việc xử lý bất đồng bộ là rất quan trọng. Dart cung cấp các công cụ mạnh mẽ để hỗ trợ điều này, bao gồm:

Future: Đại diện cho một giá trị sẽ có trong tương lai

async/await: Giúp viết code bất đồng bộ dễ hiểu hơn

Stream: Xử lý dữ liệu theo luồng (real-time)

Ví dụ, khi gọi API thời tiết, ứng dụng sẽ sử dụng async/await để lấy dữ liệu mà không làm treo giao diện.

1.7.6. Quản lý bộ nhớ và hiệu năng

Dart sử dụng cơ chế Garbage Collection để tự động giải phóng bộ nhớ không còn sử dụng. Điều này giúp giảm tải cho lập trình viên và tránh các lỗi liên quan đến rò rỉ bộ nhớ.

Ngoài ra, Dart được tối ưu để hoạt động hiệu quả trên các thiết bị di động, đảm bảo ứng dụng chạy mượt mà và ổn định.

1.7.7. Thư viện và hệ sinh thái Dart

❖ Dart cung cấp một hệ sinh thái thư viện phong phú, hỗ trợ nhiều chức năng như:

HTTP: Gửi và nhận dữ liệu từ API

JSON: Xử lý dữ liệu dạng JSON

SQLite: Kết nối và thao tác với cơ sở dữ liệu

UI: Xây dựng giao diện thông qua Flutter

Ngoài ra, Dart có kho thư viện pub.dev, nơi cung cấp hàng nghìn package hỗ trợ lập trình viên phát triển ứng dụng nhanh hơn.

1.7.8. Ứng dụng Dart trong đề tài

Xây dựng logic xử lý dữ liệu thời tiết

Gọi API và xử lý dữ liệu JSON

Quản lý trạng thái ứng dụng

Kết nối và thao tác với SQLite

Điều khiển giao diện thông qua Flutter

Dart đóng vai trò là ngôn ngữ trung tâm, giúp kết nối tất cả các thành phần của hệ thống lại với nhau.

1.7.9. Kết luận

Dart là một ngôn ngữ lập trình hiện đại, mạnh mẽ và phù hợp cho việc phát triển ứng dụng di động. Với cú pháp đơn giản, hỗ trợ đa mô hình lập trình và khả năng tích hợp chặt chẽ với Flutter, Dart là lựa chọn tối ưu cho đề tài xây dựng ứng dụng dự báo thời tiết.

1.8. Cơ sở dữ liệu SQLite

SQLite là một hệ quản trị cơ sở dữ liệu quan hệ (Relational Database Management System – RDBMS) nhẹ, được sử dụng phổ biến trong các ứng dụng di động và nhúng.

Không giống như các hệ quản trị cơ sở dữ liệu truyền thống như MySQL hay PostgreSQL, SQLite không yêu cầu cài đặt máy chủ riêng biệt mà hoạt động trực tiếp trên file dữ liệu, giúp giảm thiểu chi phí và đơn giản hóa quá trình triển khai.

1.8.1. Tổng quan về SQLite

SQLite được phát triển với mục tiêu cung cấp một giải pháp lưu trữ dữ liệu nhỏ gọn, hiệu quả và dễ sử dụng. Dữ liệu trong SQLite được lưu trữ trong một file duy nhất trên thiết bị, giúp dễ dàng quản lý và sao lưu.



Hình 1. 7: Cơ sở dữ liệu SQLite

❖ Một số đặc điểm nổi bật của SQLite:

Không cần cài đặt server, hoạt động độc lập

Dữ liệu lưu trữ trong file duy nhất

Nhẹ, nhanh và tiêu tốn ít tài nguyên

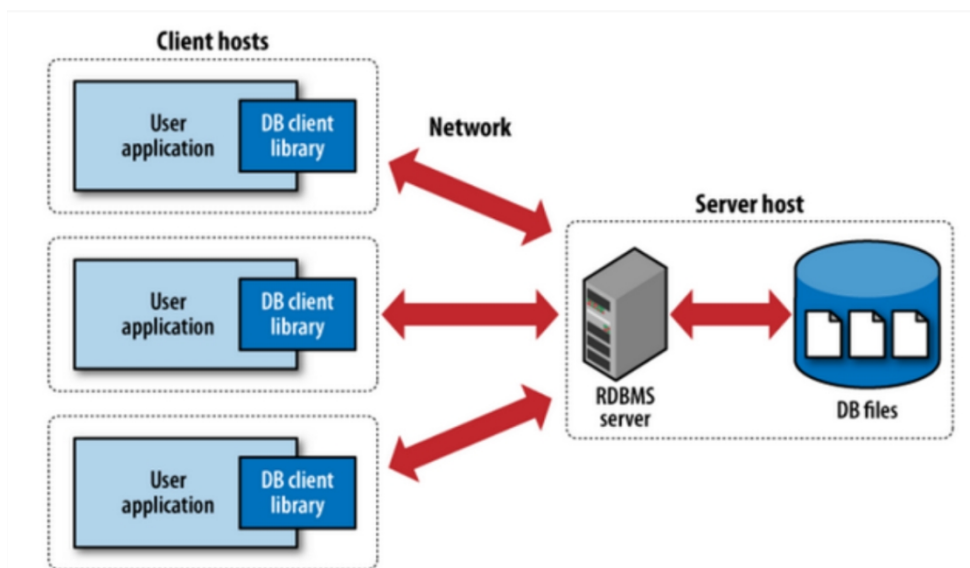
Hỗ trợ chuẩn SQL (Structured Query Language)

Tích hợp sẵn trong hầu hết các hệ điều hành di động

Nhờ những ưu điểm này, SQLite trở thành lựa chọn phù hợp cho các ứng dụng mobile như ứng dụng dự báo thời tiết trong đề tài.

1.8.2. Kiến trúc hoạt động của SQLite

SQLite hoạt động theo mô hình nhúng (embedded database), trong đó cơ sở dữ liệu được tích hợp trực tiếp vào ứng dụng. Không có sự phân tách giữa client và server như các hệ thống database truyền thống.



Hình 1. 8: Kiến trúc hoạt động của SQLite

❖ Kiến trúc của SQLite bao gồm:

SQL Compiler: Phân tích và chuyển đổi câu lệnh SQL thành mã máy

Virtual Machine (VM): Thực thi các câu lệnh SQL

B-Tree Engine: Quản lý dữ liệu trong file

File Storage: Lưu trữ dữ liệu trên thiết bị

Nhờ kiến trúc này, SQLite có thể xử lý dữ liệu nhanh chóng mà không cần kết nối mạng.

1.8.3. Các thành phần chính trong SQLite

- ❖ SQLite sử dụng mô hình dữ liệu quan hệ, bao gồm các thành phần cơ bản như:

Table (Bảng): Lưu trữ dữ liệu dưới dạng hàng và cột

Row (Bản ghi): Một dòng dữ liệu trong bảng

Column (Cột): Thuộc tính của dữ liệu

Primary Key: Khóa chính, định danh duy nhất cho mỗi bản ghi

Foreign Key: Khóa ngoại, liên kết giữa các bảng

1.8.4. Các thao tác cơ bản với SQLite

- ❖ SQLite hỗ trợ đầy đủ các thao tác cơ bản trong SQL, bao gồm:

CREATE: Tạo bảng

INSERT: Thêm dữ liệu

SELECT: Truy vấn dữ liệu

UPDATE: Cập nhật dữ liệu

DELETE: Xóa dữ liệu

1.8.5. Ưu điểm và nhược điểm của SQLite

- ❖ Ưu điểm:

Nhẹ, dễ sử dụng, không cần cài đặt phức tạp

Hiệu năng cao với dữ liệu nhỏ và trung bình

Phù hợp cho ứng dụng mobile

Không phụ thuộc vào mạng

- ❖ Nhược điểm:

Không phù hợp với hệ thống lớn, nhiều người dùng

Hạn chế về khả năng mở rộng

Không hỗ trợ tốt cho các thao tác đồng thời phức tạp

Tuy nhiên, với phạm vi của đề tài, SQLite hoàn toàn đáp ứng tốt yêu cầu lưu

trữ dữ liệu.

1.8.6. Tích hợp SQLite trong Flutter

❖ Trong Flutter, SQLite thường được sử dụng thông qua các package như:

sqflite: Thư viện phổ biến để làm việc với SQLite

path_provider: Xác định đường dẫn lưu file database

❖ Quy trình sử dụng SQLite trong Flutter:

Khởi tạo database

Tạo bảng

Thực hiện các thao tác CRUD (Create, Read, Update, Delete)

Đóng database

Việc tích hợp SQLite giúp ứng dụng hoạt động ổn định ngay cả khi không có kết nối Internet.

1.8.7. Ứng dụng SQLite trong đề tài

Lưu trữ dữ liệu thời tiết đã truy vấn

Lưu lịch sử tìm kiếm của người dùng

Lưu các cảnh báo thiên tai

Hỗ trợ truy xuất dữ liệu nhanh khi offline

Việc sử dụng SQLite giúp cải thiện hiệu năng ứng dụng và nâng cao trải nghiệm người dùng.

1.8.8. Kết luận

SQLite là một hệ quản trị cơ sở dữ liệu nhẹ, hiệu quả và phù hợp cho các ứng dụng di động. Với khả năng lưu trữ dữ liệu cục bộ, tốc độ truy xuất nhanh và dễ tích hợp, SQLite đóng vai trò quan trọng trong việc xây dựng ứng dụng dự báo thời tiết.

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Bài toán

Hiện nay, nhu cầu theo dõi thông tin thời tiết của người dùng ngày càng trở nên cần thiết trong đời sống hằng ngày. Thời tiết có ảnh hưởng trực tiếp đến việc học tập, làm việc, di chuyển, sản xuất và các hoạt động sinh hoạt cá nhân. Vì vậy, người dùng cần một công cụ có khả năng cung cấp thông tin thời tiết một cách nhanh chóng, chính xác, dễ sử dụng và có thể truy cập mọi lúc trên thiết bị di động.

Ứng dụng dự báo thời tiết được xây dựng nhằm hỗ trợ người dùng tra cứu các thông tin cơ bản về tình hình thời tiết tại một khu vực hoặc thành phố cụ thể. Thông qua ứng dụng, người dùng có thể xem nhiệt độ hiện tại, độ ẩm, tốc độ gió, trạng thái thời tiết và dự báo trong vài ngày tiếp theo. Bên cạnh đó, ứng dụng còn cho phép người dùng tìm kiếm thời tiết theo tên thành phố, lưu lại các khu vực thường xuyên theo dõi và thiết lập thông báo nhắc nhở dự báo thời tiết.

Từ những nhu cầu trên, bài toán đặt ra là xây dựng một ứng dụng di động dự báo thời tiết có giao diện trực quan, thao tác đơn giản, dữ liệu hiển thị rõ ràng và có khả năng cung cấp thông tin thời tiết kịp thời cho người dùng. Hệ thống cần đảm bảo các chức năng chính như xem thời tiết hiện tại, xem dự báo thời tiết trong tương lai, tìm kiếm thành phố, quản lý danh sách thành phố đã lưu và thiết lập thông báo dự báo thời tiết. Đây là cơ sở để xác định các yêu cầu chức năng và yêu cầu phi chức năng của hệ thống trong các phần tiếp theo.

2.1.1. Yêu cầu chức năng

Ứng dụng dự báo thời tiết cần đáp ứng các yêu cầu chức năng cơ bản nhằm hỗ trợ người dùng tra cứu và theo dõi thông tin thời tiết một cách thuận tiện. Các chức năng được xây dựng tập trung vào việc cung cấp dữ liệu thời tiết nhanh chóng, trực quan và phù hợp với nhu cầu sử dụng trên thiết bị di động.

Trước hết, hệ thống cần có chức năng **tìm kiếm thành phố**, cho phép người dùng nhập tên thành phố hoặc khu vực cần tra cứu. Sau khi người dùng thực hiện tìm kiếm, ứng dụng sẽ lấy dữ liệu thời tiết tương ứng và hiển thị kết quả trên giao diện. Chức năng này giúp người dùng có thể chủ động theo dõi thời tiết tại nhiều địa điểm khác nhau.

Bên cạnh đó, ứng dụng cần có chức năng **hiển thị thời tiết hiện tại**. Thông tin hiển thị bao gồm nhiệt độ, độ ẩm, tốc độ gió, trạng thái thời tiết và các thông tin liên quan khác. Đây là chức năng quan trọng giúp người dùng nhanh chóng nắm bắt tình hình thời tiết tại thời điểm sử dụng ứng dụng.

Ngoài việc hiển thị thời tiết hiện tại, hệ thống cũng cần cung cấp chức năng **hiển thị dự báo thời tiết** trong các ngày tiếp theo. Chức năng này giúp người dùng có cái nhìn tổng quan về xu hướng thời tiết trong tương lai, từ đó có thể chủ động sắp xếp lịch trình học tập, làm việc, di chuyển hoặc các hoạt động cá nhân.

Ứng dụng cũng cần hỗ trợ hiển thị **biểu tượng thời tiết theo trạng thái thực tế**. Tùy theo điều kiện thời tiết như trời nắng, nhiều mây, mưa, giông bão hoặc thời tiết lạnh, hệ thống sẽ hiển thị các icon tương ứng. Việc sử dụng biểu tượng trực quan giúp giao diện sinh động hơn và giúp người dùng dễ dàng nhận biết tình trạng thời tiết mà không cần đọc quá nhiều thông tin.

Ngoài ra, hệ thống cần có chức năng **lưu lịch sử tìm kiếm hoặc lưu thành phố yêu thích**. Chức năng này cho phép người dùng xem lại các thành phố đã tra cứu hoặc lưu những địa điểm thường xuyên theo dõi. Nhờ đó, người dùng không cần nhập lại tên thành phố nhiều lần, góp phần nâng cao trải nghiệm sử dụng ứng dụng.

Cuối cùng, ứng dụng cần có chức năng **làm mới dữ liệu thời tiết**. Khi người dùng cần cập nhật thông tin mới nhất, hệ thống sẽ thực hiện tải lại dữ liệu từ nguồn cung cấp thời tiết và hiển thị kết quả mới trên giao diện. Chức năng này giúp đảm bảo thông tin thời tiết luôn được cập nhật, hạn chế tình trạng dữ liệu cũ hoặc không còn phù hợp với thực tế.

2.1.2. Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, ứng dụng dự báo thời tiết cần đáp ứng các yêu cầu phi chức năng nhằm đảm bảo chất lượng sử dụng, hiệu quả vận hành và khả năng mở rộng trong tương lai. Đây là những yêu cầu không trực tiếp thể hiện qua từng chức năng cụ thể, nhưng có vai trò quan trọng trong việc nâng cao trải nghiệm người dùng và độ tin cậy của hệ thống.

Trước hết, ứng dụng cần có **giao diện dễ sử dụng**, bố cục rõ ràng và thân thiện với người dùng. Các thông tin quan trọng như nhiệt độ, độ ẩm, tốc độ gió, trạng thái

thời tiết và dự báo các ngày tiếp theo cần được sắp xếp hợp lý, giúp người dùng dễ dàng quan sát và thao tác. Giao diện cần phù hợp với nhiều nhóm người dùng, kể cả những người không có nhiều kinh nghiệm sử dụng ứng dụng công nghệ.

Thứ hai, hệ thống cần đảm bảo **tốc độ tải nhanh**. Khi người dùng tìm kiếm thành phố hoặc làm mới dữ liệu thời tiết, ứng dụng cần phản hồi trong thời gian ngắn, hạn chế tình trạng chờ đợi lâu. Điều này giúp quá trình tra cứu thông tin thời tiết diễn ra thuận tiện, đặc biệt trong các trường hợp người dùng cần xem nhanh thông tin trước khi di chuyển hoặc làm việc ngoài trời.

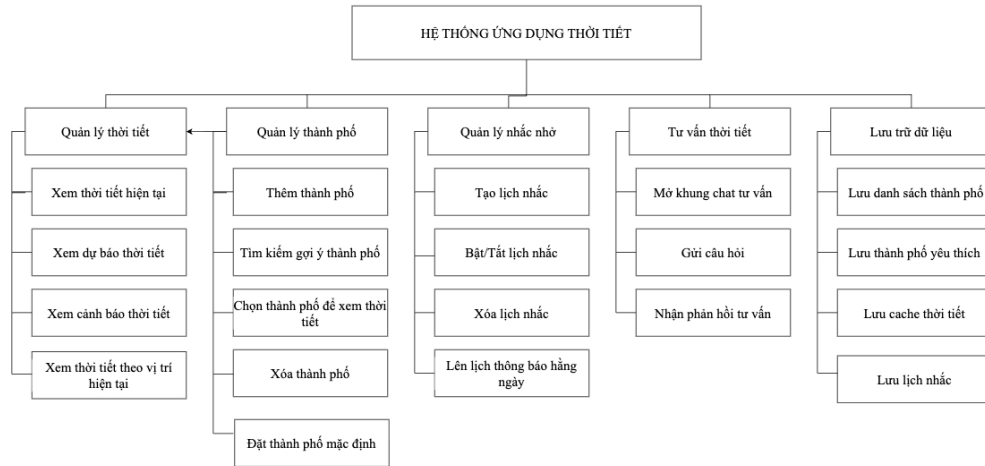
Ngoài ra, dữ liệu thời tiết cần được **hiển thị rõ ràng, dễ đọc và dễ hiểu**. Các thông tin như nhiệt độ, độ ẩm, tốc độ gió, trạng thái thời tiết và dự báo cần được trình bày bằng chữ, số và biểu tượng phù hợp. Việc hiển thị trực quan giúp người dùng nhanh chóng nắm bắt tình hình thời tiết mà không cần mất nhiều thời gian phân tích.

Ứng dụng cũng cần đảm bảo **hoạt động ổn định** trong quá trình sử dụng. Hệ thống cần hạn chế lỗi thoát ứng dụng đột ngột, lỗi không tải được dữ liệu hoặc lỗi hiển thị sai thông tin. Trong trường hợp mất kết nối mạng hoặc không lấy được dữ liệu thời tiết, ứng dụng cần có thông báo phù hợp để người dùng hiểu nguyên nhân và có thể thao tác lại.

Cuối cùng, ứng dụng cần có **khả năng triển khai trên nhiều nền tảng**. Với việc sử dụng Flutter, hệ thống có thể được phát triển để chạy trên Android và có khả năng mở rộng sang các nền tảng khác như iOS hoặc Web trong tương lai. Điều này giúp ứng dụng có tính linh hoạt cao, dễ mở rộng phạm vi sử dụng và phù hợp với xu hướng phát triển phần mềm đa nền tảng hiện nay.

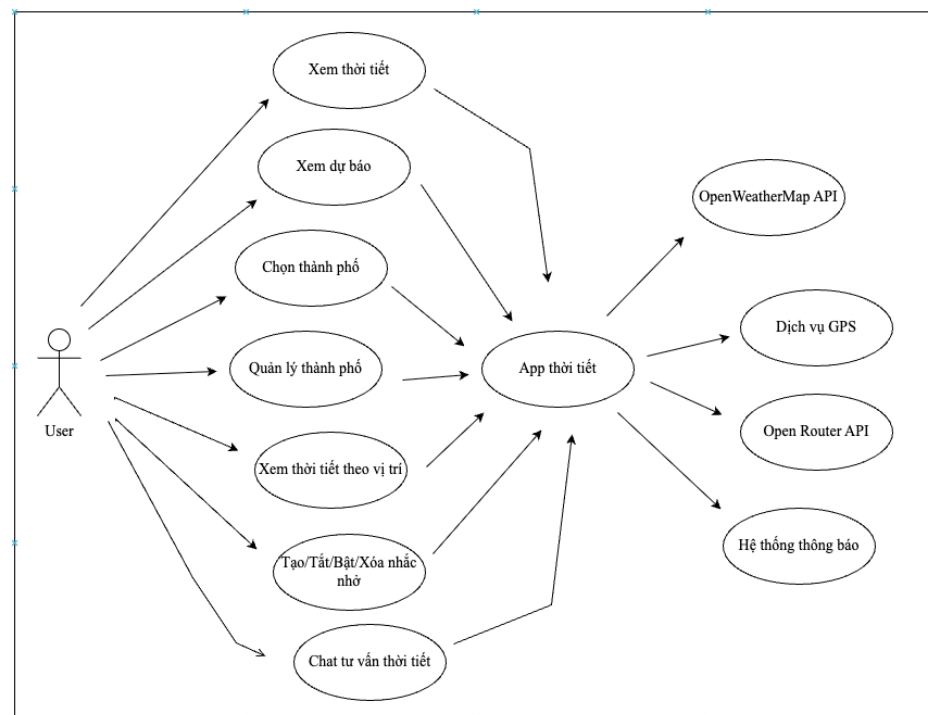
2.2. Phân tích thiết kế hệ thống

2.2.1. Sơ đồ phân cấp chức năng



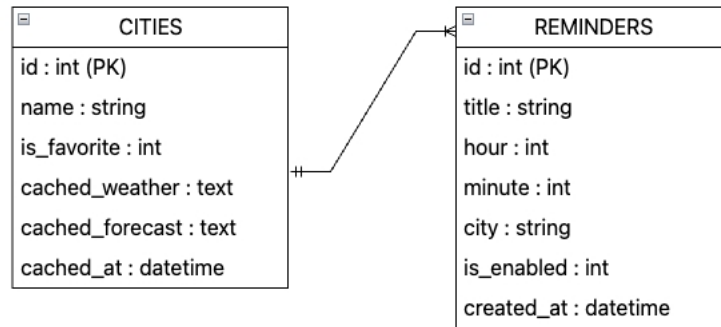
Hình 2. 1: Sơ đồ phân cấp chức năng

2.2.2. Sơ đồ Use Case



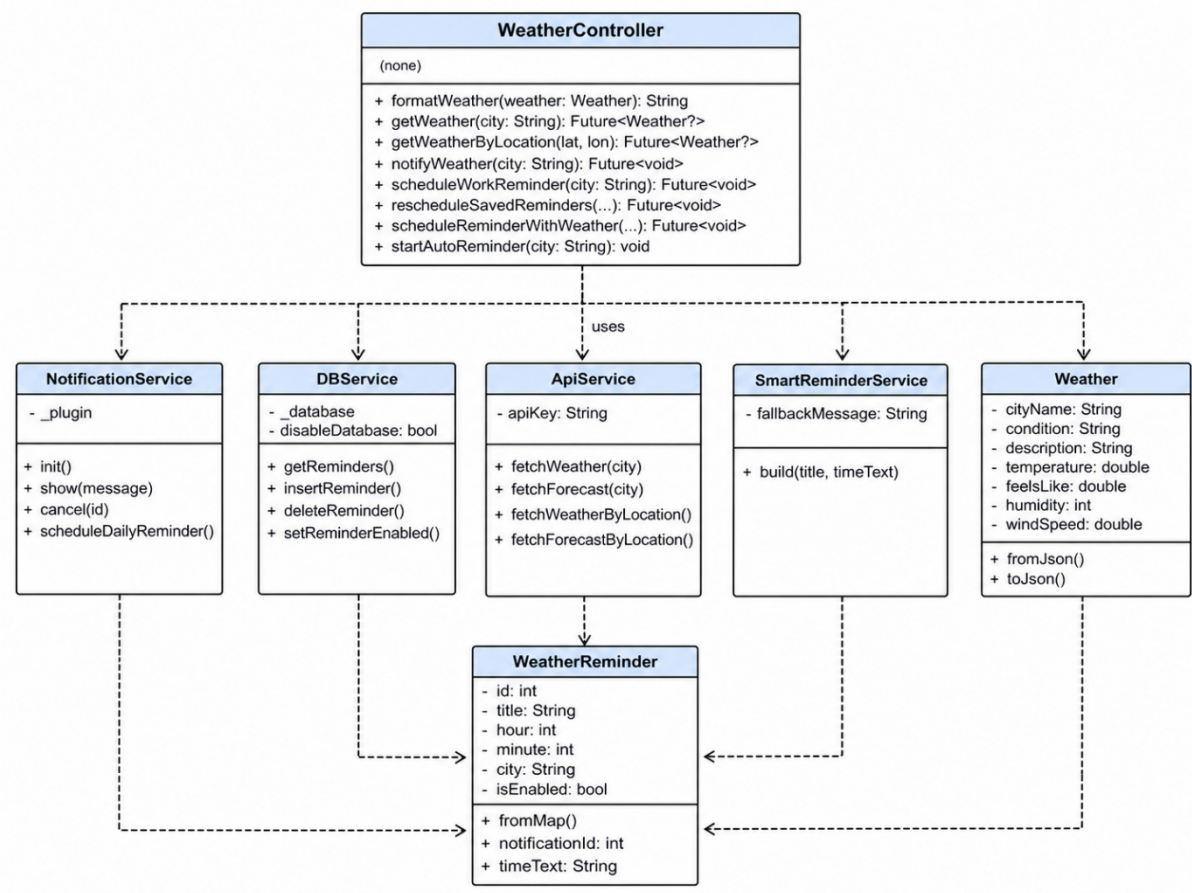
Hình 2. 2: Sơ đồ Use case

2.2.3. Sơ đồ Erd



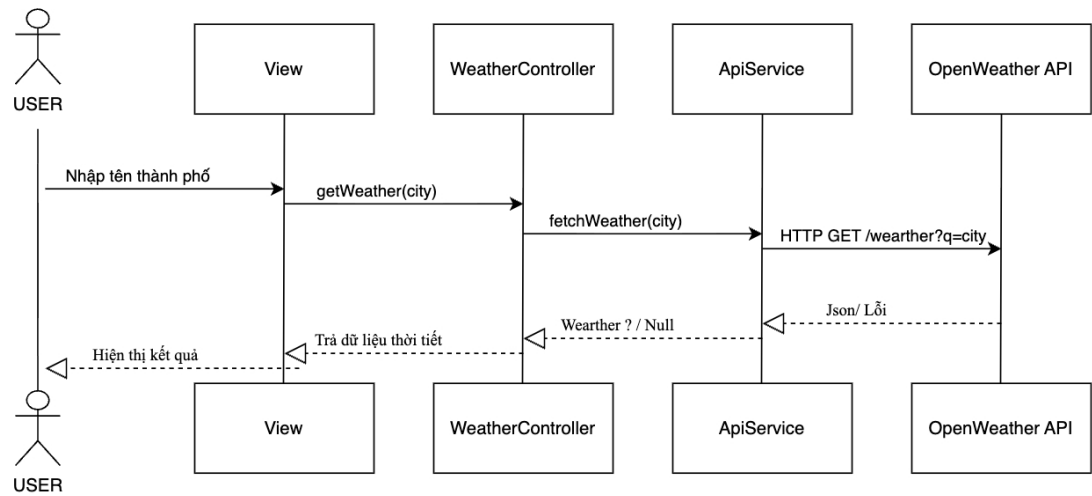
Hình 2. 3: Sơ đồ ERD

2.2.4. Sơ đồ lớp

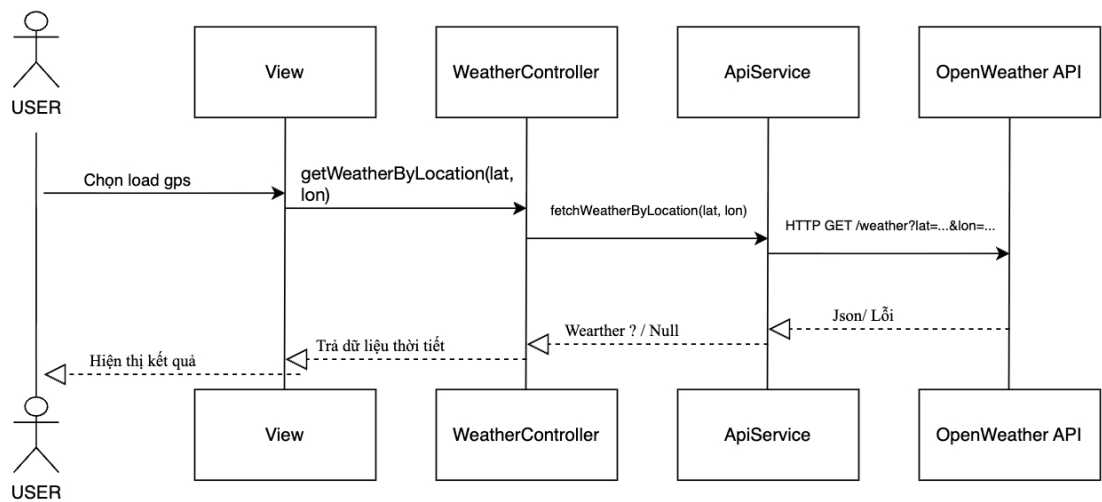


Hình 2. 4: Sơ đồ lớp

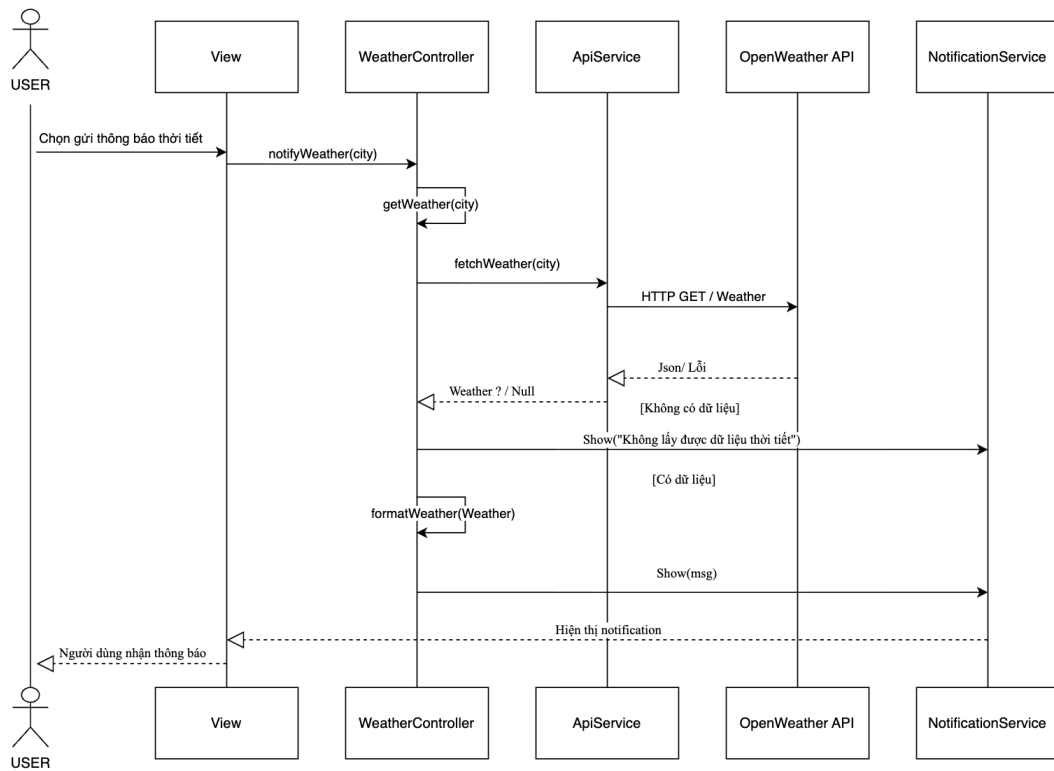
2.2.5. Sơ đồ thuần tự



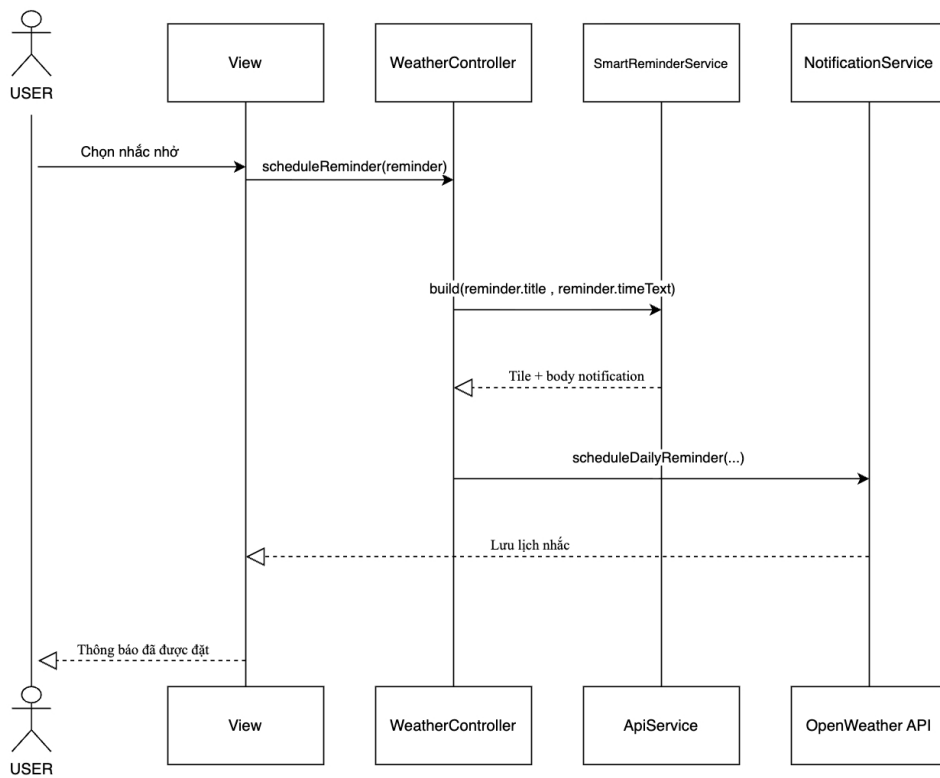
Hình 2. 5: Biểu đồ thuần tự chọn thành phố



Hình 2. 6: Biểu đồ thuần tự load vị trí hiện tại

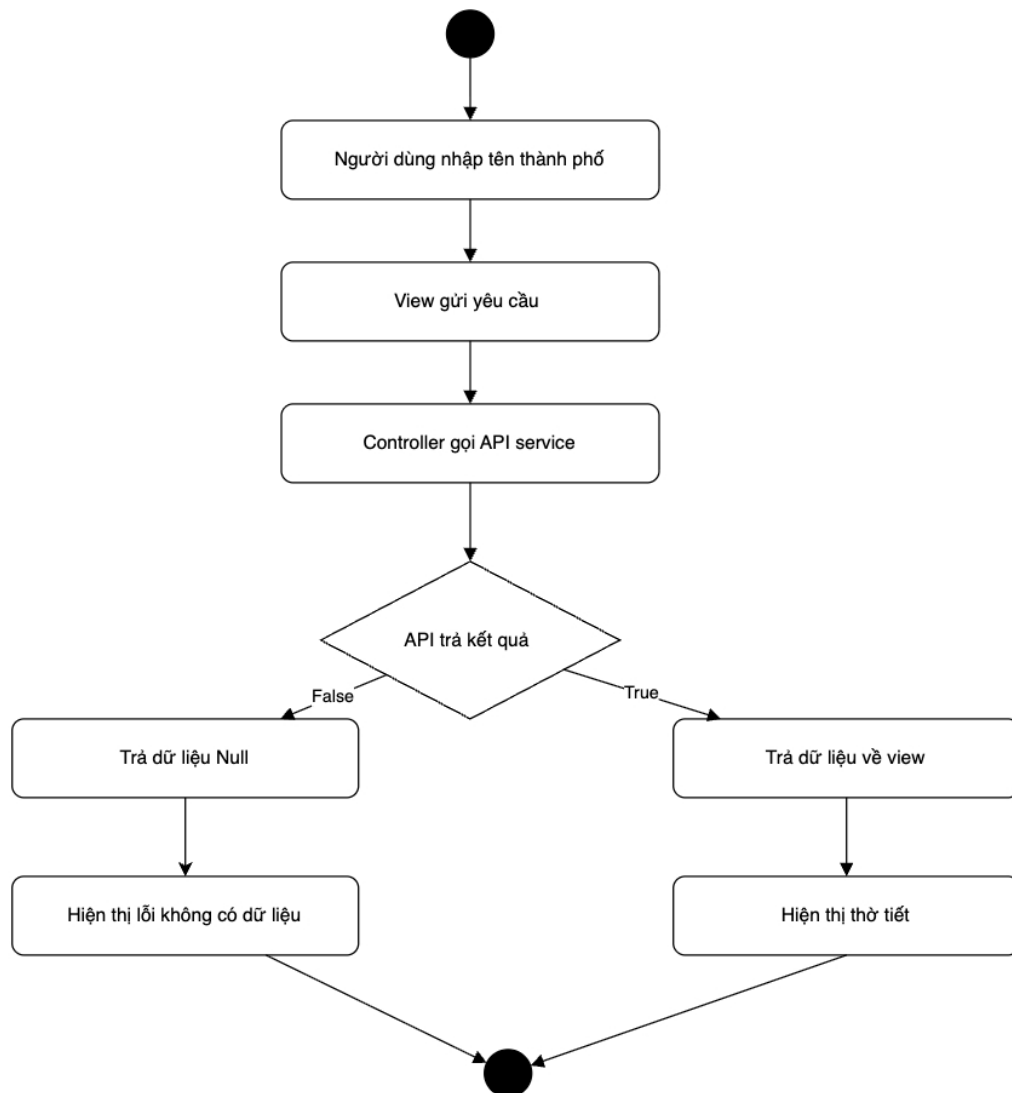


Hình 2. 7: Biểu đồ hoạt động thông báo thời tiết



Hình 2. 8: Biểu đồ tuần tự đặt lịch nhắc nhở

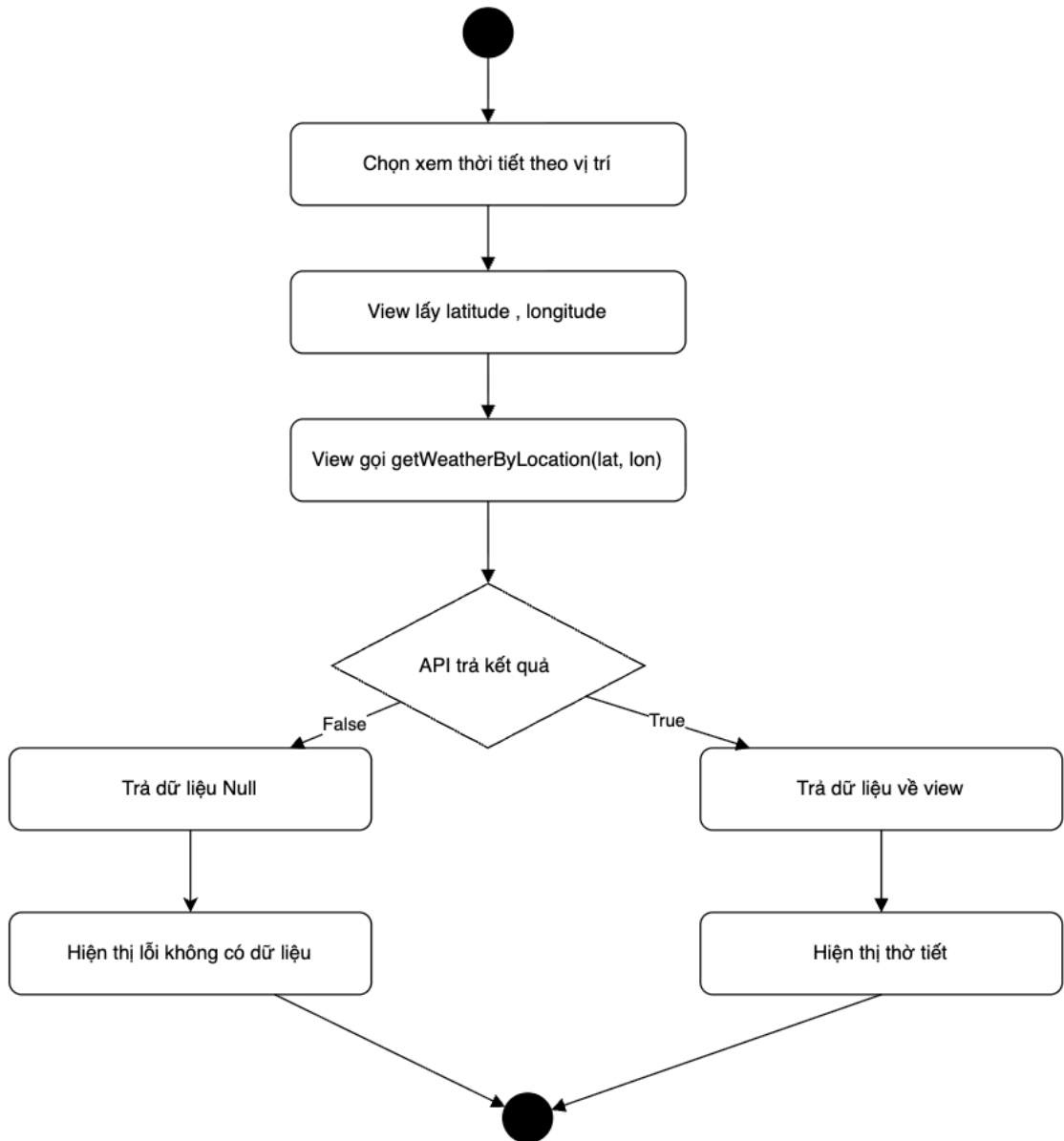
2.2.6. Biểu đồ hoạt động



Hình 2. 9: Biểu đồ hoạt động chọn thành phố

Luồng xử lý chính: Biểu đồ mô tả trình tự các bước hệ thống thực hiện khi người dùng nhập tên thành phố, từ lúc gửi yêu cầu từ giao diện đến khi Controller gọi dịch vụ API và nhận về kết quả để xử lý tiếp theo. Điều này thể hiện luồng điều khiển tuần tự của hệ thống trong xử lý yêu cầu người dùng.

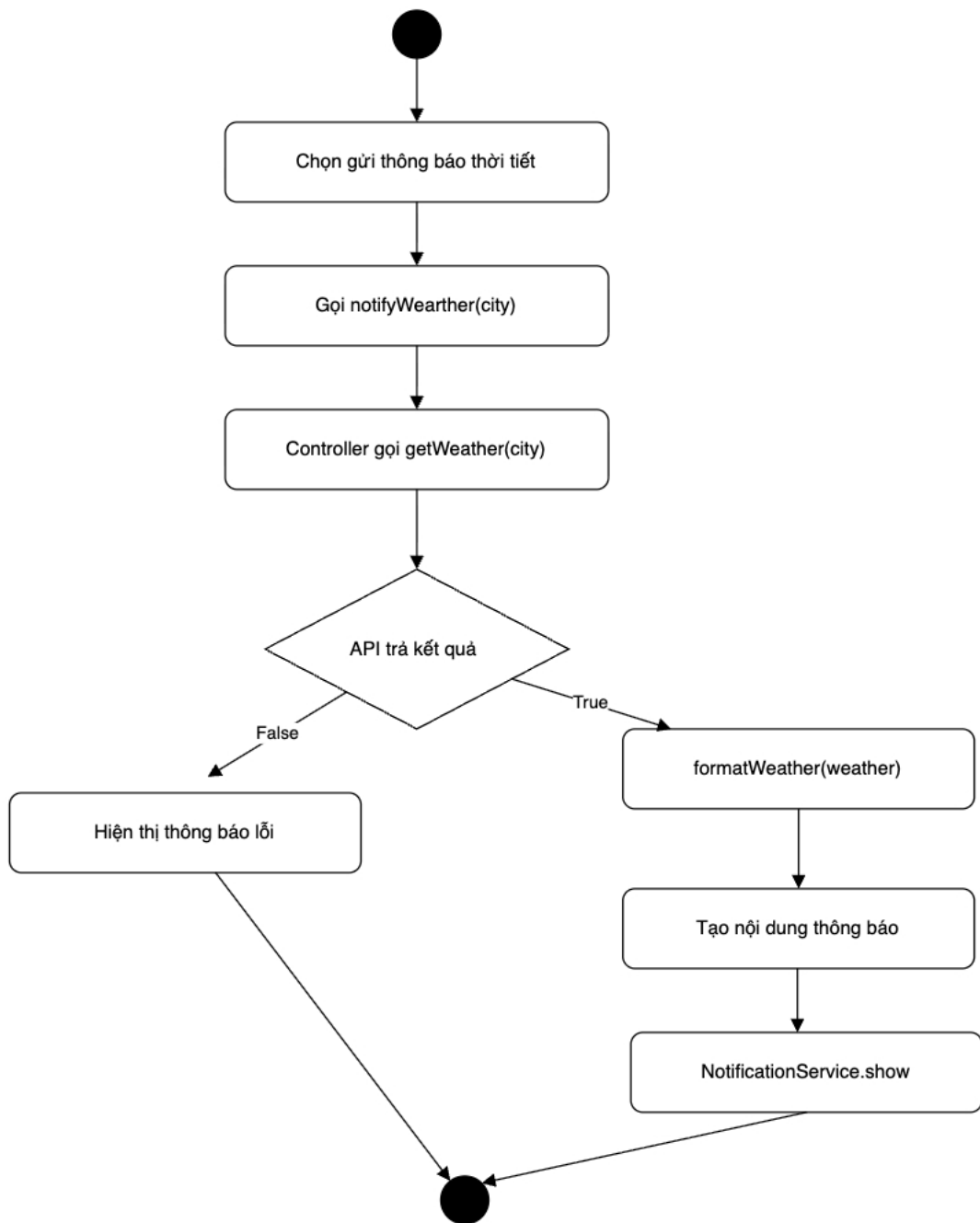
Quyết định và kết quả: Tại điểm ra quyết định, nếu API trả kết quả thành công, dữ liệu thời tiết sẽ được trả về và hiển thị; nếu không, hệ thống sẽ trả về dữ liệu null và hiện thông báo lỗi cho người dùng.



Hình 2. 10: Biểu đồ hoạt động xem vị trí hiện tại

Luồng xử lý chính: Biểu đồ hoạt động mô tả trình tự các bước khi người dùng chọn xem thời tiết theo vị trí hiện tại, từ việc bắt đầu tại nút khởi đầu → lấy tọa độ vị trí (latitude, longitude) → gọi hàm `getWeatherByLocation(lat, lon)` để gửi yêu cầu tới API → sau đó điều hướng theo từng nhánh tùy thuộc vào kết quả trả về của API.

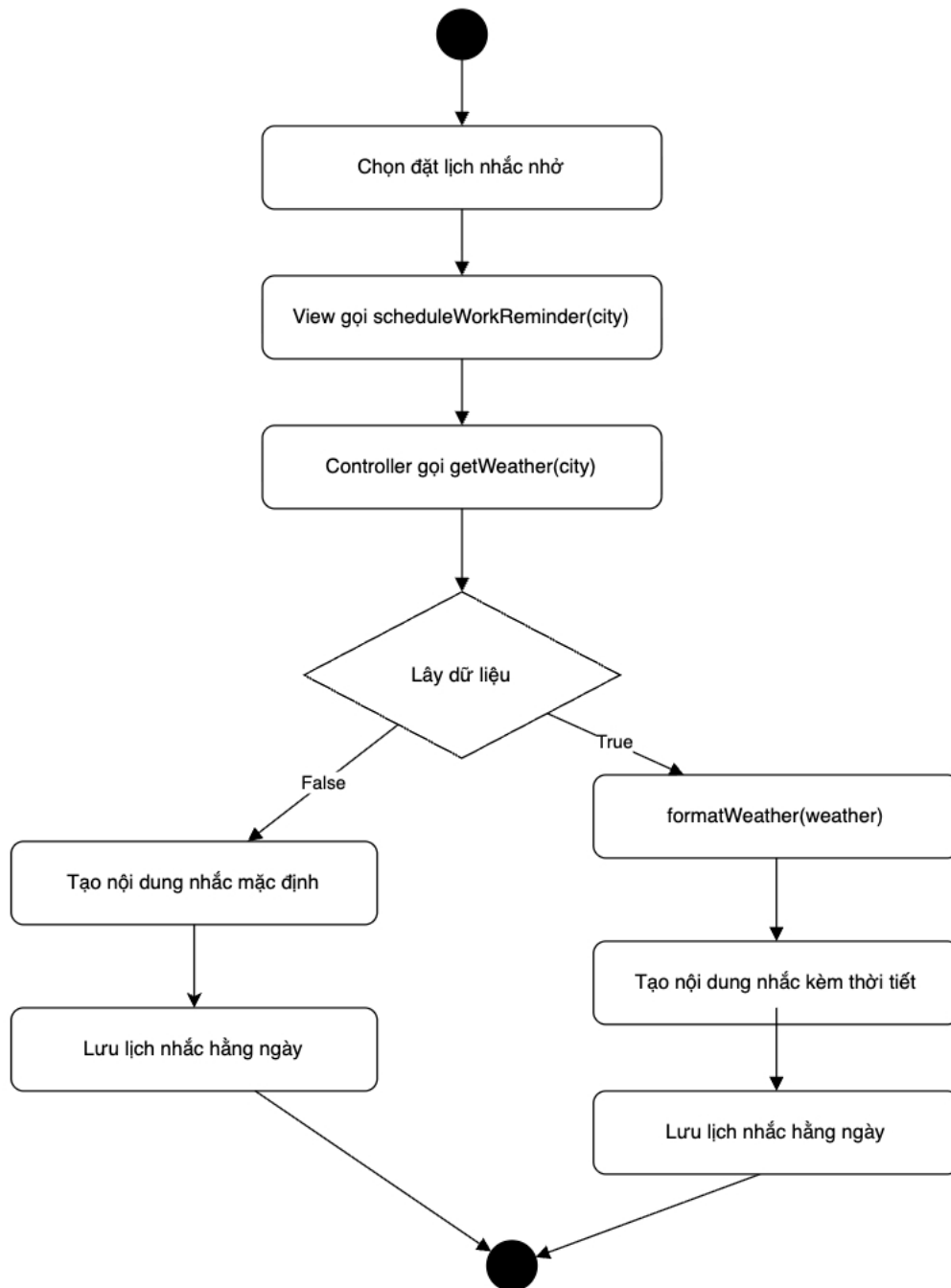
Quyết định và kết quả: Tại nút quyết định, nếu API trả về dữ liệu thành công thì hệ thống trả dữ liệu về View và hiện thị thông tin thời tiết; nếu API trả null (không có dữ liệu) thì hệ thống hiển thị thông báo lỗi không có dữ liệu trước khi kết thúc luồng hoạt động.



Hình 2. 11: Biểu đồ hoạt động gửi thông báo thời tiết

Luồng xử lý chính: Biểu đồ thể hiện trình tự các bước khi người dùng chọn gửi thông báo thời tiết: từ thao tác người dùng → gọi hàm → Controller lấy dữ liệu API → quyết định dựa trên kết quả trả về API cho tới bước tiếp theo.

Quyết định và kết quả: Tại điểm quyết định nếu API trả thành công thì dữ liệu được xử lý và hiển thị thông báo; nếu thất bại thì hệ thống dừng và hiển thị thông báo lỗi phù hợp.

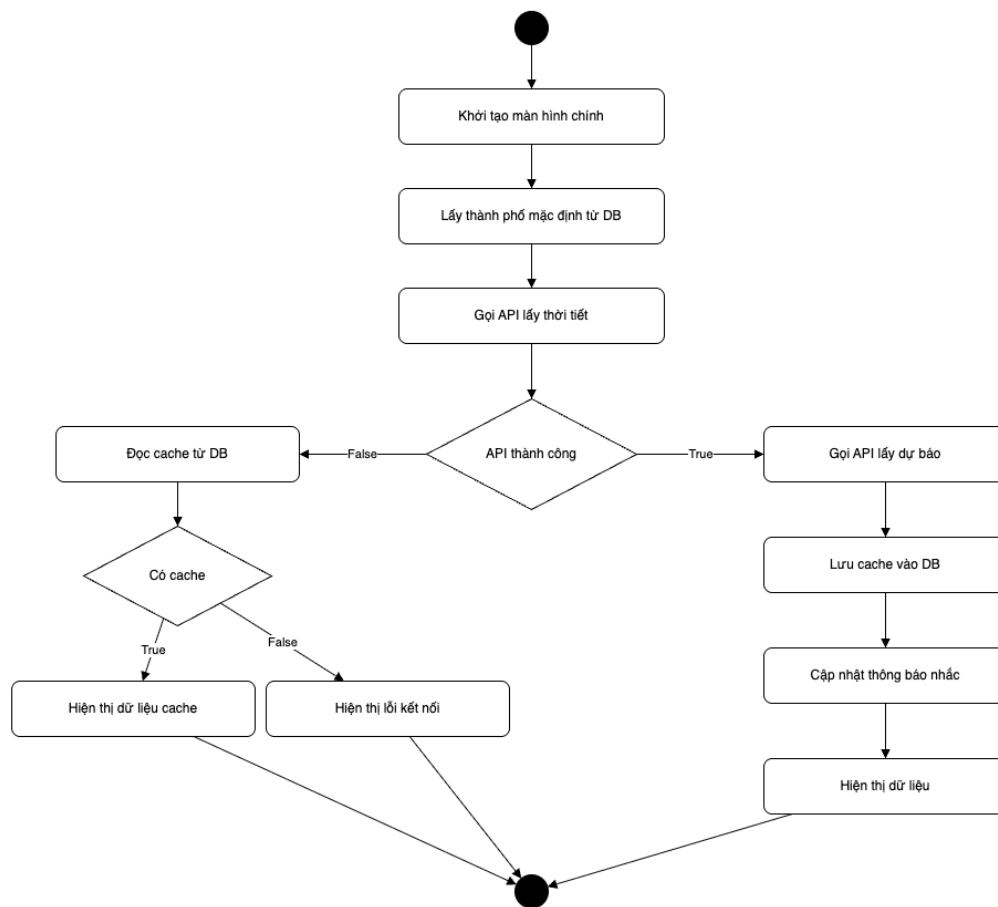


Hình 2. 12: Biểu đồ hoạt động đặt lịch nhắc nhở

Luồng xử lý chính: Biểu đồ hoạt động minh họa các bước khi người dùng đặt lịch nhắc nhở thời tiết: bắt đầu từ việc người dùng chọn đặt lịch → View gọi hàm → Controller gọi API để lấy dữ liệu thời tiết → sau đó điều hướng theo kết quả trả về của API, thể hiện trình tự xử lý từ đầu đến cuối.

Quyết định và kết quả: Tại điểm quyết định, nếu API trả dữ liệu thành công thì hệ thống định dạng dữ liệu và tạo nội dung nhắc nhở rồi lưu lịch nhắc hàng ngày; nếu

API không trả dữ liệu (False) thì hệ thống tạo nhắc nhở mặc định và lưu lịch nhắc hàng ngày trước khi kết thúc luồng.



Hình 2.13: Biểu đồ hoạt động tổng quát

Luồng xử lý chính: Biểu đồ hoạt động trong Hình 2.13 mô tả trình tự xử lý khi khởi tạo màn hình chính của ứng dụng, từ lúc lấy thành phố mặc định từ cơ sở dữ liệu → gọi API để lấy thời tiết → theo kết quả thành công hay không API điều hướng các bước tiếp theo trong quy trình hoạt động.

Quyết định và kết quả: Tại nút quyết định, nếu API trả về thành công (True), hệ thống lưu cache và cập nhật dữ liệu thời tiết, sau đó hiển thị dữ liệu mới cho người dùng; nếu API thất bại (False), hệ thống đọc cache từ cơ sở dữ liệu rồi hiển thị dữ liệu đã lưu nếu có, hoặc hiển thị lỗi kết nối nếu không có cache.

2.3. Cài đặt kiểm thử

Sau khi hoàn thành quá trình phân tích và thiết kế, nhóm tiến hành cài đặt và

kiểm thử ứng dụng nhằm đánh giá khả năng hoạt động của hệ thống trong môi trường thực tế. Việc cài đặt kiểm thử giúp xác định mức độ đáp ứng của ứng dụng đối với các yêu cầu đã đặt ra, đồng thời phát hiện và xử lý các lỗi có thể phát sinh trong quá trình sử dụng.

❖ Môi trường cài đặt

Ứng dụng được xây dựng và kiểm thử trong môi trường phát triển phù hợp với nền tảng Flutter. Cụ thể, nhóm sử dụng **Flutter SDK** để phát triển ứng dụng di động đa nền tảng, kết hợp với ngôn ngữ lập trình **Dart** để xây dựng giao diện, xử lý logic và quản lý dữ liệu trong ứng dụng. Công cụ **Android Studio** được sử dụng làm môi trường phát triển chính, hỗ trợ viết mã nguồn, chạy giả lập Android và kiểm tra hoạt động của ứng dụng trên thiết bị mô phỏng.

Bên cạnh đó, hệ thống sử dụng **API dự báo thời tiết** để lấy dữ liệu thời tiết theo thời gian thực. API này đóng vai trò là nguồn cung cấp thông tin về nhiệt độ, độ ẩm, tốc độ gió, trạng thái thời tiết và dự báo trong các ngày tiếp theo. Dữ liệu sau khi được lấy từ API sẽ được xử lý và hiển thị trên giao diện ứng dụng.

❖ Các giao diện được xây dựng

Trong quá trình cài đặt, nhóm đã xây dựng các giao diện chính phục vụ cho nhu cầu sử dụng của người dùng. **Màn hình chính** là nơi hiển thị thông tin thời tiết hiện tại, bao gồm nhiệt độ, trạng thái thời tiết, độ ẩm, tốc độ gió và các thông tin liên quan. Đây là giao diện trung tâm giúp người dùng nhanh chóng theo dõi tình hình thời tiết tại thành phố đã chọn.

Màn hình tìm kiếm cho phép người dùng nhập tên thành phố cần tra cứu. Sau khi người dùng thực hiện tìm kiếm, ứng dụng sẽ gửi yêu cầu đến API thời tiết, nhận dữ liệu trả về và hiển thị kết quả tương ứng. Giao diện này giúp người dùng dễ dàng xem thông tin thời tiết của nhiều địa điểm khác nhau.

Ngoài ra, ứng dụng còn có **màn hình danh sách thành phố đã lưu**, dùng để hiển thị các thành phố mà người dùng đã tìm kiếm hoặc lưu lại để theo dõi thường xuyên. Chức năng này giúp người dùng tiết kiệm thời gian khi không cần nhập lại tên thành phố nhiều lần.

Bên cạnh các màn hình trên, nhóm cũng xây dựng **màn hình thao tác nhắc nhở**. Giao diện này cho phép người dùng thiết lập hoặc kiểm tra chức năng thông báo dự báo thời tiết. Thông qua đó, ứng dụng có thể hỗ trợ người dùng nhận thông tin thời tiết vào thời điểm phù hợp.

❖ Nội dung kiểm thử

Quá trình kiểm thử được thực hiện dựa trên các tình huống sử dụng phổ biến của người dùng. Trước hết, nhóm tiến hành kiểm thử chức năng **nhập đúng tên thành phố**. Khi người dùng nhập một thành phố hợp lệ, ứng dụng cần lấy được dữ liệu từ API và hiển thị đầy đủ thông tin thời tiết tương ứng trên màn hình.

Tiếp theo, nhóm kiểm tra tình huống **kết nối mạng không ổn định hoặc không có mạng**. Trong trường hợp này, ứng dụng cần hiển thị thông báo phù hợp để người dùng biết rằng dữ liệu không thể được tải do lỗi kết nối. Điều này giúp tránh việc ứng dụng bị treo hoặc hiển thị giao diện trống gây khó hiểu cho người dùng.

Ngoài ra, nhóm cũng kiểm thử trường hợp **API gặp lỗi hoặc không trả về dữ liệu hợp lệ**. Khi xảy ra lỗi từ phía API, ứng dụng cần xử lý ngoại lệ, không bị thoát đột ngột và đưa ra thông báo rõ ràng để người dùng có thể thử lại sau.

Cuối cùng, nhóm kiểm thử chức năng **thông báo dự báo thời tiết** bằng cách bấm nút kiểm tra thông báo. Khi thao tác được thực hiện, ứng dụng cần hiển thị thông báo thử nghiệm đúng nội dung, đúng thời điểm và đảm bảo người dùng có thể nhận được cảnh báo từ hệ thống.

Thông qua quá trình cài đặt và kiểm thử, ứng dụng cơ bản đáp ứng được các chức năng chính như xem thời tiết hiện tại, tìm kiếm thành phố, lưu thành phố đã tra cứu, làm mới dữ liệu và kiểm tra thông báo. Kết quả kiểm thử là cơ sở để nhóm tiếp tục hoàn thiện, tối ưu giao diện và nâng cao độ ổn định của ứng dụng trong các giai đoạn phát triển tiếp theo.

2.4. Publication

Sau khi hoàn thành quá trình xây dựng và kiểm thử cơ bản, nhóm tiến hành đóng gói ứng dụng thành file cài đặt APK để triển khai thử nghiệm trên thiết bị Android. Việc đóng gói APK giúp ứng dụng có thể được cài đặt trực tiếp trên điện

thoại, từ đó kiểm tra khả năng hoạt động của hệ thống trong môi trường sử dụng thực tế thay vì chỉ chạy trên trình giả lập.

Trong quá trình publication, ứng dụng được build từ mã nguồn Flutter sang định dạng APK dành cho hệ điều hành Android. Sau khi quá trình đóng gói hoàn tất, file APK được cài đặt lên điện thoại Android để thử nghiệm các chức năng chính như xem thời tiết hiện tại, tìm kiếm thành phố, hiển thị dự báo thời tiết, làm mới dữ liệu và kiểm tra thông báo dự báo thời tiết.

Việc cài đặt trên thiết bị thật giúp nhóm đánh giá chính xác hơn về tốc độ tải dữ liệu, độ ổn định của ứng dụng, khả năng hiển thị giao diện trên màn hình điện thoại và khả năng nhận thông báo. Qua đó, nhóm có thể phát hiện thêm các vấn đề phát sinh trong quá trình sử dụng thực tế để tiếp tục điều chỉnh và hoàn thiện ứng dụng.

CHƯƠNG 3: XÂY DỰNG ỨNG DỤNG

3.1. Môi trường phát triển

Ứng dụng được phát triển trên nền tảng Flutter, sử dụng ngôn ngữ lập trình Dart với bộ SDK 3.11.3. Môi trường lập trình chính là Android Studio. Ứng dụng hướng đến nền tảng Android, phục vụ chức năng xem thời tiết, dự báo, nhắc nhở và lưu trữ dữ liệu cục bộ. Các thư viện chính được sử dụng gồm:

http: gọi API thời tiết từ OpenWeatherMap

sqflite: lưu trữ dữ liệu cục bộ bằng SQLite

path: hỗ trợ tạo đường dẫn file cơ sở dữ liệu

geolocator: lấy vị trí GPS hiện tại của người dùng

flutter_local_notifications: hiển thị và đặt lịch thông báo cục bộ

timezone: hỗ trợ xử lý múi giờ khi hẹn thông báo

3.2. Cài đặt và cấu hình hệ thống

Đầu tiên, nhóm tạo project Flutter và cài đặt các thư viện cần thiết trong file pubspec.yaml. Sau đó chạy lệnh flutter pub get để tải dependency. Hệ thống được cấu hình theo các bước:

Khai báo các package phục vụ API, SQLite, GPS và thông báo.

Khởi tạo NotificationService trong hàm main() trước khi chạy giao diện.

Thiết lập múi giờ Asia/Ho_Chi_Minh để lịch nhắc hoạt động chính xác.

Cấu hình quyền truy cập vị trí và quyền gửi thông báo trên thiết bị Android.

Tạo giao diện chính HomeView làm màn hình khởi động của ứng dụng.

3.3. Xây dựng các chức năng chính

Sau khi hoàn tất cấu hình hệ thống, nhóm tiến hành xây dựng các chức năng chính của ứng dụng, bao gồm hiển thị thời tiết hiện tại, dự báo thời tiết, cảnh báo thời tiết, lưu trữ dữ liệu bằng SQLite và tích hợp API thời tiết.

3.3.1. *Hiển thị thời tiết hiện tại*

Chức năng hiển thị thời tiết hiện tại cho phép người dùng nhập tên thành phố hoặc chọn vị trí hiện tại để xem thông tin thời tiết. Khi người dùng gửi yêu cầu, giao diện gọi WeatherController, sau đó controller tiếp tục gọi ApiService để lấy dữ liệu từ OpenWeatherMap.

Dữ liệu nhận về được chuyển thành đối tượng Weather thông qua phương thức fromJson(). Sau đó giao diện hiển thị các thông tin quan trọng như tên thành phố, trạng thái thời tiết, nhiệt độ, độ ẩm và tốc độ gió.

Trong trường hợp không lấy được dữ liệu từ API, hệ thống sẽ kiểm tra dữ liệu cache trong SQLite để tiếp tục hiển thị cho người dùng.

3.3.2. Dự báo thời tiết

Bên cạnh thời tiết hiện tại, ứng dụng còn cung cấp chức năng dự báo thời tiết trong các mốc thời gian tiếp theo. Controller sử dụng phương thức getForecast(city) hoặc getForecastByLocation(latitude, longitude) để lấy danh sách dự báo từ API.

Dữ liệu dự báo được trả về dưới dạng danh sách các mốc thời gian, mỗi mốc gồm nhiệt độ, trạng thái thời tiết, độ ẩm và gió. Giao diện sẽ hiển thị danh sách này theo từng khung giờ, đồng thời cho phép người dùng xem chi tiết từng bản ghi dự báo.

3.3.3. Cảnh báo thời tiết

Ứng dụng tích hợp chức năng cảnh báo thời tiết nguy hiểm nhằm hỗ trợ người dùng chủ động hơn khi ra ngoài. Chức năng này được xây dựng trong phương thức getWeatherAlert(weather, forecast) của WeatherController.

Hệ thống kiểm tra các điều kiện như giông sét, mưa lớn, nắng nóng gay gắt hoặc gió mạnh từ dữ liệu thời tiết hiện tại. Ngoài ra, chương trình còn phân tích 6 mốc dự báo gần nhất để phát hiện nguy cơ mưa hoặc thời tiết xấu trong vài giờ tới. Nếu phát hiện rủi ro, hệ thống sẽ trả về chuỗi cảnh báo phù hợp để hiển thị cho người dùng.

3.3.4. Lưu trữ dữ liệu bằng SQLite

Để tăng khả năng hoạt động khi mất kết nối mạng, ứng dụng sử dụng SQLite để lưu trữ dữ liệu cục bộ. Thành phần DBService chịu trách nhiệm tạo và quản lý cơ sở dữ liệu weather.db. Hệ thống lưu các thông tin như:

Danh sách thành phố đã tìm kiếm

Thành phố yêu thích

Dữ liệu cache thời tiết và dự báo

Danh sách reminder do người dùng tạo

Khi API không phản hồi hoặc thiết bị mất mạng, ứng dụng sẽ đọc dữ liệu cache từ SQLite và hiển thị lại cho người dùng. Nhờ đó trải nghiệm sử dụng không bị gián đoạn.

3.3.5. Tích hợp API thời tiết

Ứng dụng sử dụng API của OpenWeatherMap để lấy dữ liệu thời tiết hiện tại, dự báo thời tiết và thông tin theo tọa độ GPS. Toàn bộ phần gọi mạng được đóng gói trong lớp ApiService. Các phương thức chính gồm:

fetchWeather(city): lấy thời tiết hiện tại theo tên thành phố

fetchForecast(city): lấy dự báo thời tiết

fetchWeatherByLocation(latitude, longitude): lấy thời tiết theo GPS

fetchForecastByLocation(latitude, longitude): lấy dự báo theo GPS

Sau khi gửi HTTP request, hệ thống kiểm tra mã phản hồi. Nếu thành công, dữ liệu JSON sẽ được phân tích và chuyển thành model phù hợp. Nếu thất bại, chương trình trả về null hoặc danh sách rỗng để controller xử lý tiếp.

3.4. Xử lý dữ liệu và logic chương trình

Logic xử lý chính của ứng dụng được tổ chức theo mô hình phân tách giữa giao diện, controller, model và service. Trong đó, HomeView đảm nhiệm hiển thị giao diện và nhận thao tác từ người dùng; WeatherController đóng vai trò trung gian điều phối; các service như ApiService, DBService, NotificationService và SmartReminderService phụ trách từng tác vụ chuyên biệt.

Khi người dùng yêu cầu xem thời tiết, HomeView gửi yêu cầu tới WeatherController. Controller gọi ApiService để lấy dữ liệu từ OpenWeatherMap, sau đó chuyển dữ liệu JSON về đối tượng Weather. Nếu dữ liệu lấy thành công, hệ thống tiếp tục lưu cache vào SQLite để sử dụng khi mất mạng. Nếu gọi API thất bại, chương

trình sẽ đọc dữ liệu cache từ DB và hiển thị lại cho người dùng.

Đối với chức năng thông báo, controller lấy dữ liệu thời tiết hiện tại, định dạng nội dung bằng `formatWeather()` rồi gửi sang `NotificationService` để hiển thị thông báo cục bộ. Với chức năng nhắc nhở, hệ thống sử dụng `scheduleDailyReminder()` để đặt lịch thông báo định kỳ vào thời điểm cố định. Ngoài ra, các reminder do người dùng tạo sẽ được lưu trong SQLite và tự động khôi phục khi ứng dụng tải lại dữ liệu thời tiết mới.

Việc tách riêng model, controller và service giúp mã nguồn rõ ràng hơn, dễ bảo trì và thuận tiện mở rộng thêm chức năng trong tương lai.

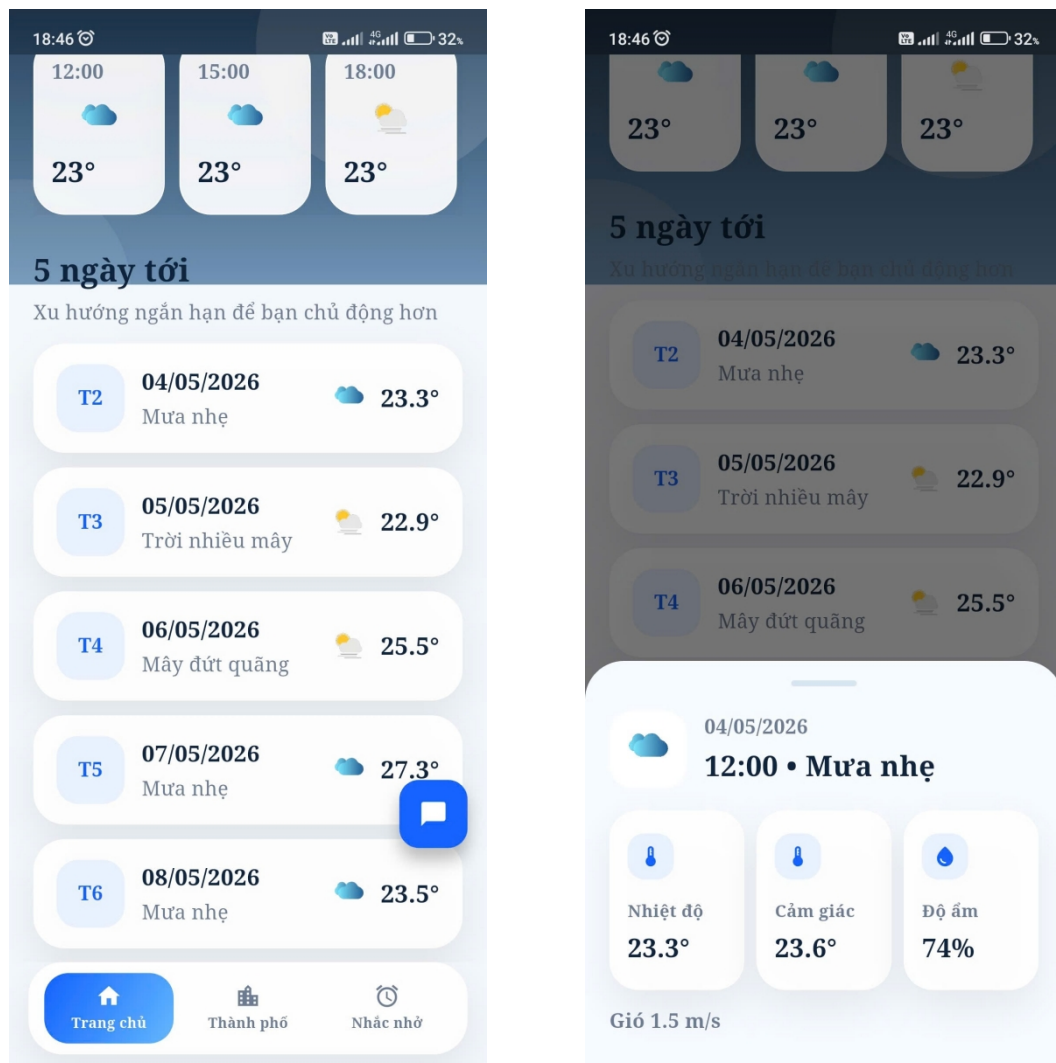
3.5. Giao diện ứng dụng



Hình 3. 1: Màn hình chính của ứng dụng part 1

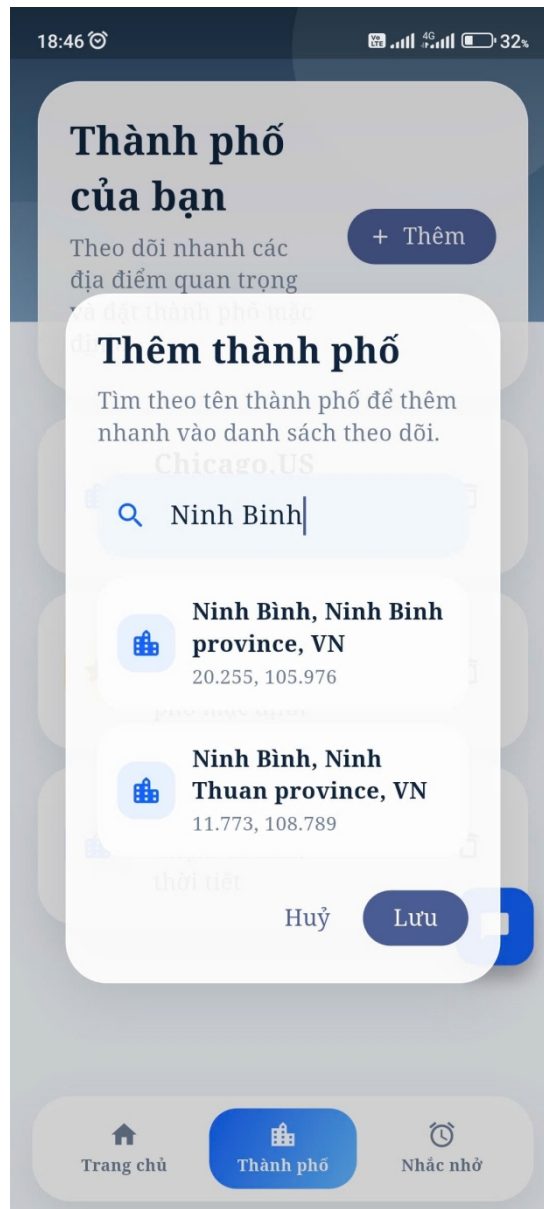
Màn hình hiển thị nhiệt độ hiện tại của thành phố được chọn. Trạng thái thời tiết (nắng, mưa, mây, v.v.) được thể hiện rõ bằng văn bản. Tên thành phố và thời gian cập nhật dữ liệu xuất hiện ở phía trên. Giao diện có phong nền thay đổi nhẹ theo tình

trạng thời tiết.



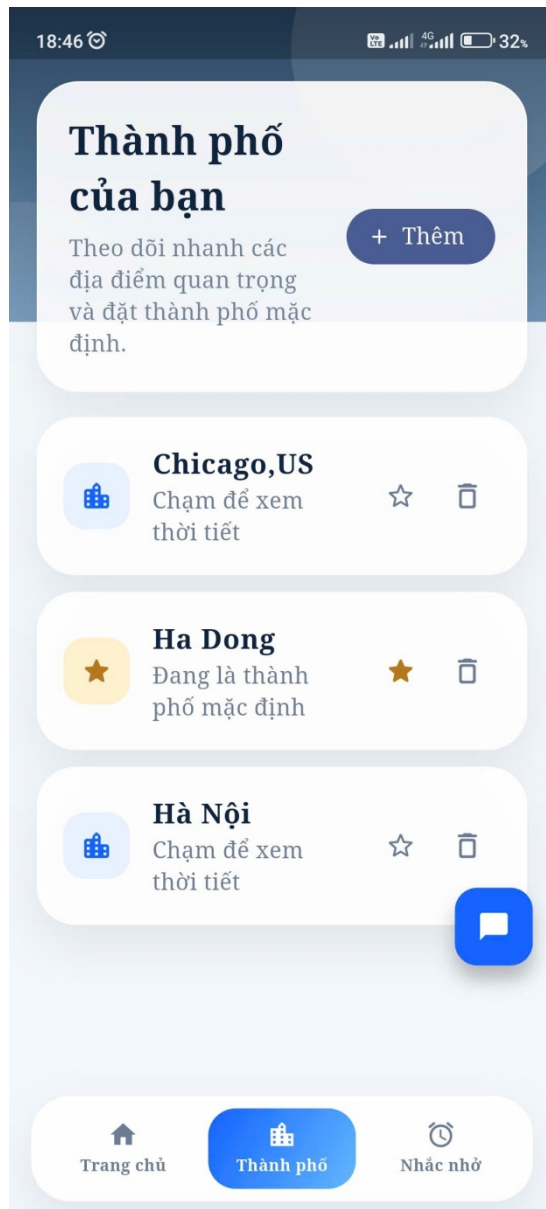
Hình 3. 2: Màn hình chính của ứng dụng part 2

Phần dưới màn hình hiển thị độ ẩm không khí theo tỷ lệ phần trăm. Tốc độ gió được trình bày kèm đơn vị m/s hoặc km/h. Biểu tượng minh họa (icon) tương ứng với thời tiết hiện tại, ví dụ ô che dù khi mưa. Bố cục cân đối, màu sắc dễ chịu, ưu tiên các chỉ số thời tiết chính.



Hình 3. 3: Màn hình tìm kiếm thành phố

Người dùng nhập tên thành phố vào ô tìm kiếm ở đầu màn hình. Ứng dụng gọi API thời tiết sau khi người dùng xác nhận hoặc chọn từ gợi ý. Kết quả trả về hiển thị ngay thông tin thời tiết của thành phố cần tra cứu.



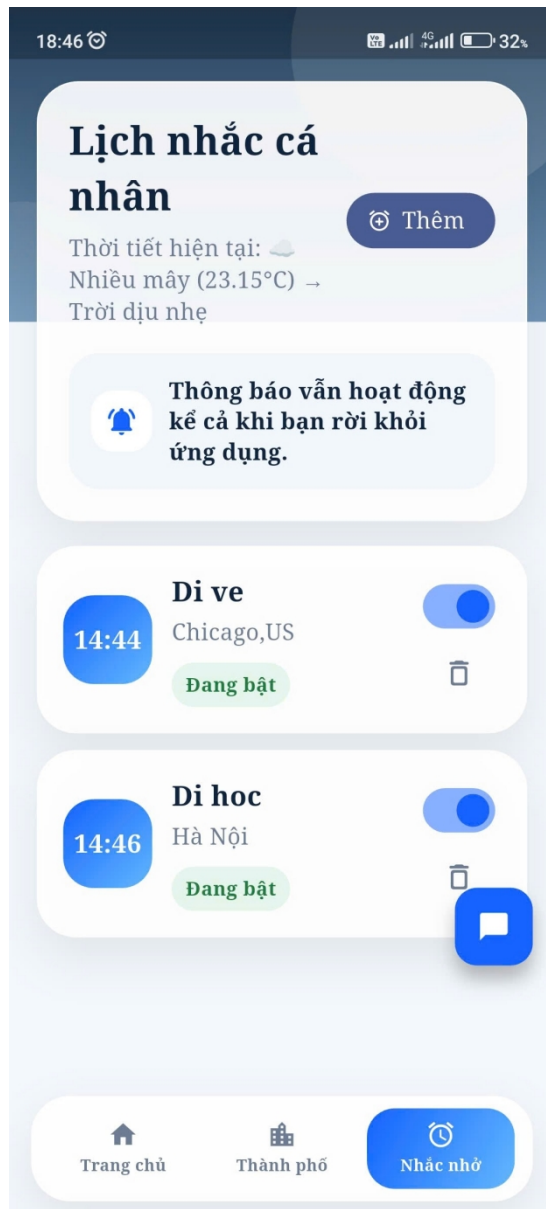
Hình 3. 4: Màn hình danh sách thành phố đã lưu

Danh sách các thành phố được lưu sau khi tìm kiếm hoặc chủ động thêm. Mỗi dòng hiển thị tên thành phố, nhiệt độ tóm tắt và biểu tượng thời tiết nhỏ. Người dùng có thể vuốt hoặc nhấn vào biểu tượng xóa để bỏ khỏi danh sách. Chạm vào một thành phố sẽ chuyển về màn hình chính với thông tin chi tiết.



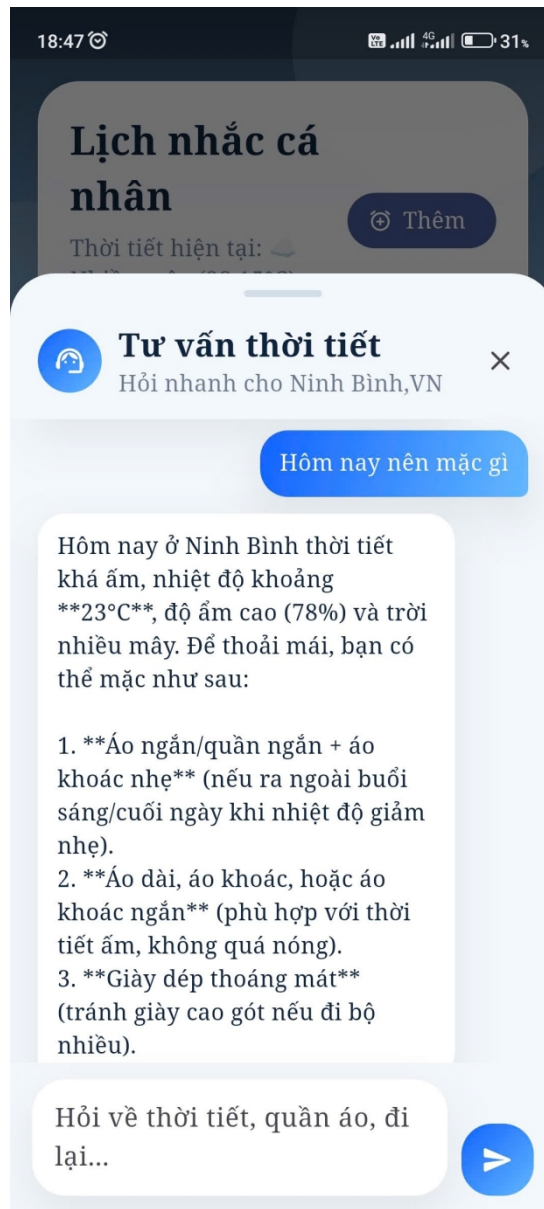
Hình 3. 5: Màn hình thiết lập nhắc nhở thời tiết

Người dùng bật/tắt công tắc để kích hoạt thông báo dự báo. Có thể chọn giờ nhận thông báo (ví dụ 7:00 sáng, 18:00 tối). Ứng dụng hỗ trợ nhắc cảnh báo thời tiết cục đoạn (bão, mưa lớn). Cài đặt được lưu cục bộ hoặc đồng bộ với tài khoản người dùng.



Hình 3. 6: Màn hình quản lý lịch nhắc cá nhân

Hiển thị danh sách các lịch nhắc do người dùng tự tạo. Mỗi nhắc nhở gồm tiêu đề, thời gian, ngày lặp lại và ghi chú liên quan đến thời tiết. Có nút thêm mới, chỉnh sửa và xóa nhắc cho từng mục. Lịch nhắc có thể kết hợp thông tin thời tiết để gợi ý mang theo ô, áo ấm, v.v.



Hình 3. 7: Màn hình giao diện chat AI

Màn hình hiển thị khung chat giữa người dùng và trợ lý AI. Người dùng gõ câu hỏi bằng ngôn ngữ tự nhiên, ví dụ: “Ngày mai Hà Nội có mưa không?” AI phản hồi nhanh bằng văn bản, kèm nhiệt độ dự báo hoặc khuyến nghị. Lịch sử hội thoại được giữ lại trong phiên, có thể xóa hoặc mới.

KẾT LUẬN

❖ Kết luận

Sau quá trình tìm hiểu, phân tích, thiết kế và cài đặt, nhóm đã xây dựng được ứng dụng dự báo thời tiết trên nền tảng di động bằng Flutter. Ứng dụng cơ bản đáp ứng được các chức năng chính đã đề ra, bao gồm xem thời tiết hiện tại, tìm kiếm thành phố, hiển thị thông tin dự báo thời tiết, hiển thị biểu tượng theo trạng thái thời tiết, lưu thành phố đã tra cứu và hỗ trợ kiểm tra thông báo dự báo thời tiết.

Ứng dụng giúp người dùng có thể nhanh chóng theo dõi các thông tin thời tiết cần thiết như nhiệt độ, độ ẩm, tốc độ gió và tình trạng thời tiết tại khu vực quan tâm. Thông qua giao diện trực quan và thao tác đơn giản, người dùng có thể dễ dàng tra cứu thông tin thời tiết phục vụ cho nhu cầu sinh hoạt, học tập, làm việc và di chuyển hằng ngày.

Tuy nhiên, bên cạnh những kết quả đạt được, ứng dụng vẫn còn tồn tại một số hạn chế nhất định. Dữ liệu thời tiết của hệ thống còn phụ thuộc vào API bên ngoài, do đó khi API gặp lỗi, thay đổi giới hạn truy cập hoặc mất kết nối mạng thì ứng dụng có thể không lấy được dữ liệu mới. Ngoài ra, giao diện của ứng dụng tuy đã đáp ứng được các chức năng cơ bản nhưng vẫn chưa được tối ưu hoàn toàn về trải nghiệm người dùng, bố cục hiển thị và mức độ trực quan trên nhiều kích thước màn hình khác nhau.

❖ Hướng phát triển

Trong thời gian tới, nhóm có thể tiếp tục phát triển và hoàn thiện ứng dụng theo nhiều hướng khác nhau. Trước hết, ứng dụng có thể được bổ sung chức năng dự báo, giúp người dùng nhận biết sớm các hiện tượng thời tiết nguy hiểm như mưa lớn, bão, giông lốc, nắng nóng hoặc rét đậm. Đây là hướng phát triển quan trọng nhằm nâng cao tính thực tiễn và giá trị sử dụng của hệ thống.

Bên cạnh đó, nhóm có thể bổ sung biểu đồ thời tiết để thể hiện sự thay đổi của nhiệt độ, độ ẩm, lượng mưa hoặc tốc độ gió theo từng giờ, từng ngày. Việc sử dụng biểu đồ sẽ giúp người dùng dễ dàng quan sát xu hướng thời tiết thay vì chỉ xem các con số riêng lẻ.

Ngoài ra, ứng dụng có thể được mở rộng thêm chức năng hỗ trợ nhiều ngôn ngữ, chẳng hạn như tiếng Việt và tiếng Anh. Điều này giúp ứng dụng tiếp cận được nhiều nhóm người dùng hơn, đồng thời nâng cao tính chuyên nghiệp và khả năng mở rộng trong tương lai.

Nhìn chung, ứng dụng đã hoàn thành được các chức năng nền tảng của một hệ thống dự báo thời tiết trên thiết bị di động. Đây là cơ sở quan trọng để nhóm tiếp tục cải tiến, bổ sung các chức năng nâng cao và phát triển ứng dụng theo hướng hoàn thiện hơn.

TÀI LIỆU THAM KHẢO

- [1] ITNavi. (n.d.). Flutter là gì?. Truy cập ngày 14 tháng 04 năm 2026, từ <https://itnavi.com.vn/blog/flutter-la-gi>
- [2] TechWorks. (n.d.). Flutter và cấu trúc của Flutter. Truy cập ngày 14 tháng 04 năm 2026, từ <https://techworks.vn/blog/flutter-va-cau-truc-cua-flutter>
- [3] Viblo. (2021). Flutter là gì? Các đặc tính vượt trội của Flutter. Truy cập ngày 14 tháng 04 năm 2026, từ <https://viblo.asia/p/flutter-la-gi-cac-dac-tinh-vuot-troi-cua-flutter-2021-RQqKL606l7z>
- [4] Viblo. (n.d.). Tìm hiểu về Flutter. Truy cập ngày 14 tháng 04 năm 2026, từ <https://viblo.asia/p/tim-hieu-ve-flutter-ORNZq9PqZ0n>
- [5] Tokyo Tech Lab. (n.d.). Ngôn ngữ Dart là gì?. Truy cập ngày 14 tháng 04 năm 2026, từ <https://tokyotechlab.com/vi/blogs/ngon-ngu-dart-la-gi>
- [6] Viblo. (n.d.). Khám phá lập trình bất đồng bộ trong Dart Flutter. Truy cập ngày 14 tháng 04 năm 2026, từ <https://viblo.asia/p/kham-pha-lap-trinh-bat-dong-bo-trong-dart-flutter-XL6lA6nB5eka>
- [7] Viblo. (n.d.). SQLite là gì?. Truy cập ngày 14 tháng 04 năm 2026, từ <https://viblo.asia/p/sqlite-la-gi-E375zVVR5GW>
- [8] Bizfly Cloud. (2024). SQLite là gì?. Truy cập ngày 14 tháng 04 năm 2026, từ <https://bizflycloud.vn/tin-tuc/sqlite-la-gi-20241023091822663.htm>
- [9] Trần, A., & Nguyễn, B. (2024). Ứng dụng công nghệ thông tin trong dự báo thời tiết và phòng tránh thiên tai. Hà Nội: Nhà xuất bản Khoa học và Kỹ thuật.
- [10] Google. (n.d.). Flutter documentation. Retrieved May 13, 2026, from <https://docs.flutter.dev/>
- [11] Intergovernmental Panel on Climate Change. (2023). Climate Change 2023: Synthesis Report. Contribution of Working Groups I, II and III to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change. IPCC.

- [12] World Meteorological Organization. (2021). WMO Guidelines on Multi-hazard Early Warning Systems. WMO-No. 1228.
- [13] Nguyễn, H., & Trần, Q. (2023). Ứng dụng công nghệ thông tin trong dự báo thời tiết và phòng tránh thiên tai. Hà Nội: Nhà xuất bản Khoa học và Kỹ thuật.
- [14] Trần, A. (2024). Biến đổi khí hậu và tác động đến hiện tượng thời tiết cực đoan. TP. Hồ Chí Minh: Nhà xuất bản Đại học Quốc gia TP. HCM.
- [15] Google. (2023). Flutter documentation. Retrieved from <https://flutter.dev/docs>
- [16] Chen, W., Liu, Y., & Zhang, X. (2020). Cross-platform mobile app development: Performance and UI consistency analysis of Flutter. *Journal of Systems and Software*, 164, 110540. <https://doi.org/10.1016/j.jss.2020.110540>
- [17] Joorabian, M., & Sadeghi, H. (2021). A review on Flutter framework for mobile applications: Design patterns and best practices. *International Journal of Advanced Computer Science and Applications*, 12(8), 389–398. <https://doi.org/10.14569/IJACSA.2021.0120848>

PHỤ LỤC

Phụ lục A. Kết quả kiểm thử

Phần này trình bày kết quả kiểm thử một số chức năng chính của ứng dụng.

STT	Chức năng kiểm thử	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Đánh giá
1	Tìm kiếm thành phố	Hà Nội	Hiển thị thời tiết Hà Nội	Hiển thị đúng dữ liệu	Đạt
2	Tìm kiếm thành phố	London	Hiển thị thời tiết London	Hiển thị đúng dữ liệu	Đạt
3	Tìm kiếm sai tên thành phố	abcxyz	Hiển thị thông báo lỗi	Hiển thị thông báo lỗi	Đạt
4	Kiểm tra mất mạng	Tắt kết nối Internet	Thông báo lỗi kết nối	Hiển thị thông báo phù hợp	Đạt
5	Làm mới dữ liệu	Bấm nút làm mới	Cập nhật dữ liệu mới	Dữ liệu được tải lại	Đạt
6	Kiểm tra thông báo	Bấm nút test thông báo	Hiển thị thông báo thử nghiệm	Thông báo hiển thị thành công	Đạt

Phụ lục B. Thông tin môi trường phát triển

Thành phần	Thông tin
Nền tảng phát triển	Flutter
Ngôn ngữ lập trình	Dart

Công cụ phát triển	Android Studio
Cơ sở dữ liệu cục bộ	SQLite
Nguồn dữ liệu	API dự báo thời tiết
Thiết bị kiểm thử	Điện thoại Android / Máy ảo Android
Định dạng đóng gói	APK